

机器视觉技术

刘晓芳

liuxiaofang@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

©孙明竹

主要内容

- 第六章 卷积神经网络
- 第七章 图像分类
- 第八章 目标检测与图像分割
 - 8.1 目标检测
 - 8.2 语义分割
 - 8.3 实例分割

8.1 目标检测

- 定位+分类，对于仅有一个目标的图片，检测出该目标所处的位置以及该目标的类别
- 目标检测：对于有多个目标的图片，检测出所有目标所处的位置及其类别



8.1.1 定位+分类

- 定位+分类问题是分类到目标检测的一个过渡问题，从单纯地图片分类到分类后给出目标所处的位置，再到多目标的类别和位置
- 定位问题则需要模型返回目标所在的外界矩形框，即目标的 (x, y, w, h) 四元组



→ CAT

a) 分类问题



→ (x, y, w, h)

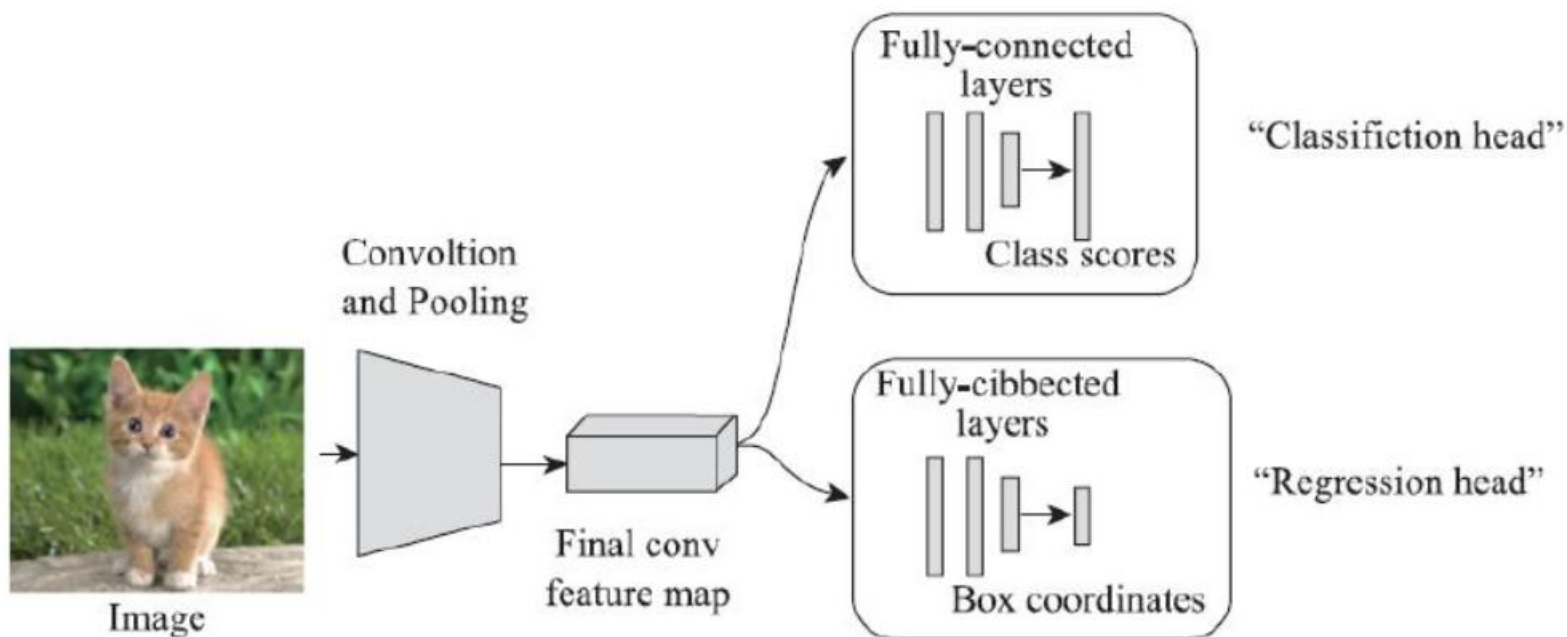
b) 定位问题

8.1.1 定位+分类

- 将定位当作回归问题
 - 训练/下载一个分类模型，如AlexNet、VGGNet或ResNet
 - 在分类网络最后一个卷积层的特征层（feature map）上添加“regression head”
 - 同时训练“classification head”和“regression head”，为了同时训练分类和定位两个问题，最终损失函数是分类和定位两个“head”产生的损失的加权和

8.1.1 定位+分类

- 在预测时同时使用分类和回归“head”得到分类+定位的结果
 - 分类预测出的结果就是C个类别
 - 回归预测的结果可能有两种：一种是类别无关，输出4个值；一种是类别相关，输出 $4 \times C$ 个值



8.1.1 定位+分类

➤ 定位+分类存在问题

- 目标检测需要获取图片中所有目标的位置及其类别，当图片中只有一个目标时，“regression head”预测4个值，当图片中有3个目标时，“regression head”预测12个值，那么当图片中有多个目标时，“regression head”具体要预测多少个值呢？



CAT:(x, y, w, h)



DOG:(x, y, w, h)
DOG:(x, y, w, h)
CAT:(x, y, w, h)

8.1 目标检测

- 尝试使用滑窗的方法来解决，需要设计大量的不同尺度和长宽比的“滑窗”来使它们通过CNN，然而这个计算量是非常巨大的
- 有没有什么方法能使我们快速定位到目标的潜在区域，从而减少大量不必要的计算呢？



Dog? NO
Cat? NO
Background? YES



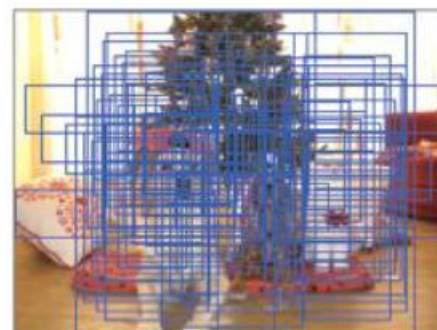
Dog? YES
Cat? NO
Background? NO



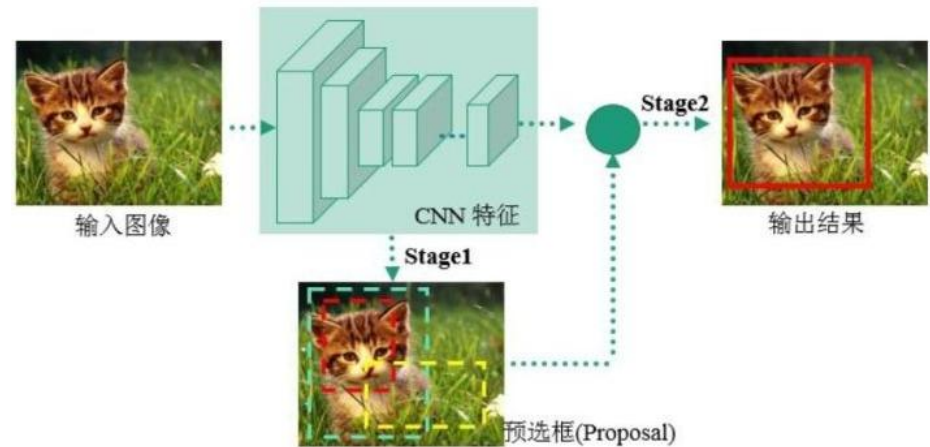
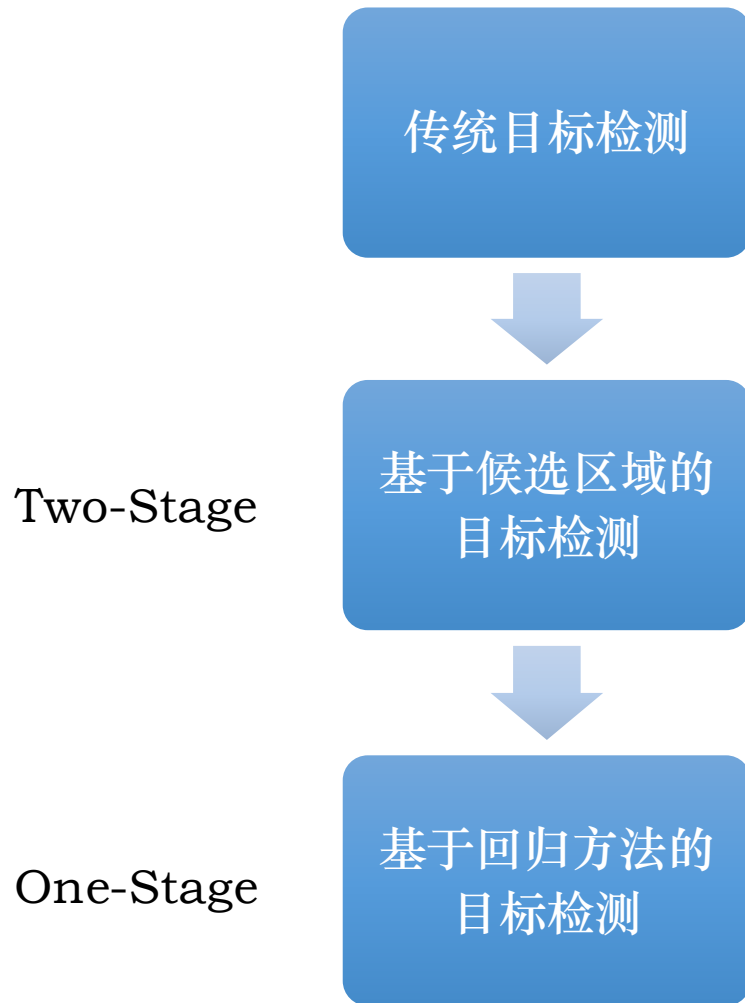
Dog? YES
Cat? NO
Background? NO



Dog? NO
Cat? YES
Background? NO

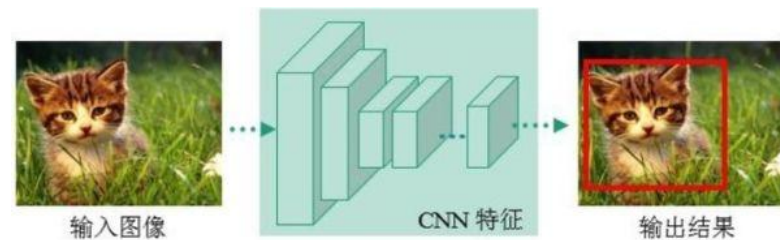


8.1 目标检测



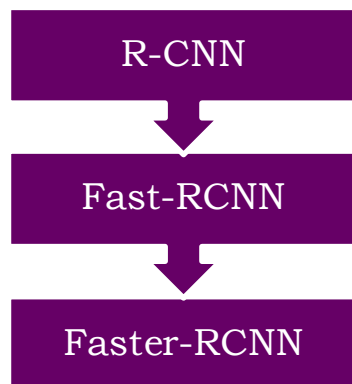
Two Stage结构

知乎 @Hero



One Stage结构

8.1.2 Two-stage目标检测方法



Rich feature hierarchies for accurate object detection and semantic segmentation

Ross Girshick Jeff Donahue Trevor Darrell Jitendra Malik
UC Berkeley

Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun

Fast R-CNN

Ross Girshick
Microsoft Research

Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun

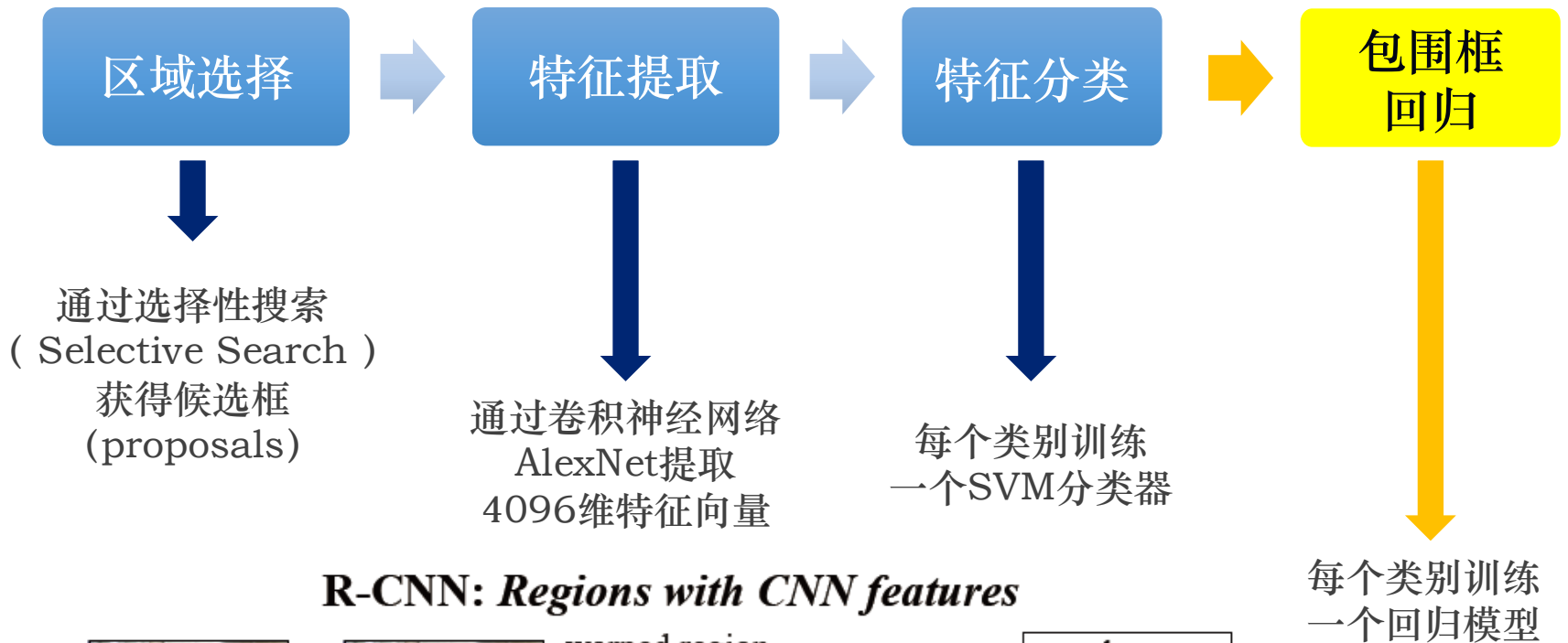
<https://arxiv.org/abs/1311.2524>

<https://arxiv.org/abs/1406.4729v4>

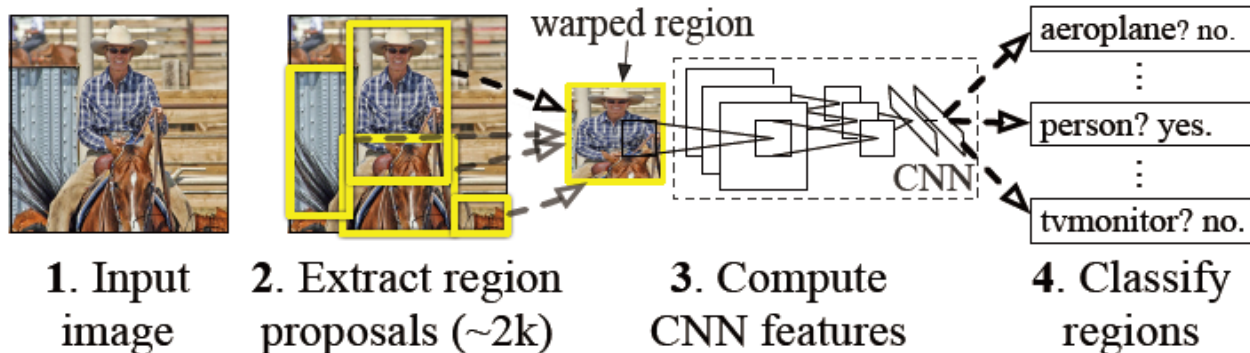
<https://arxiv.org/abs/1504.08083>

<https://arxiv.org/abs/1506.01497>

1) R-CNN



R-CNN: *Regions with CNN features*



1) R-CNN

➤ Selective search方法

- 从图片中选出潜在物体候选框 (Regions of Interest, ROI) 的方法
- 有了获取ROI的方法，接下来就可以通过分类和合并的方法来获取最终的目标检测结果了



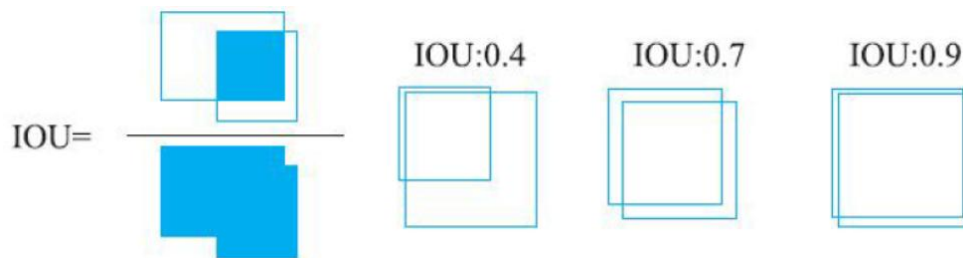
1) R-CNN

- 1. 选出潜在目标候选框 (ROI)
 - R-CNN使用selective search的方法选出了2000个潜在物体候选框
- 2. 训练一个好的特征提取器
 - 使用AlexNet提取4096维的特征向量 (VGGNet、GoogLeNet或ResNet等)
 - ROI尺寸大小不定，需要调整到指定尺寸



1) R-CNN

- 为了获得一个好的特征提取器，一般在ImageNet预训练好的模型基础上做调整，将ImageNet中的1000个类别的输出改为 $(C+1)$ 个输出，其中， C 是真实需要预测的类别个数，1是背景类
- 新特征的训练方法是使用随机梯度下降（Stochastic Gradient Descent, SGD）
- 正样本和负样本：衡量两个矩形交叠情况的指标IOU（Intersection Over Union），两个矩形面积的交集除以并集， $IOU \geq 0.5$ 时则认为是正样本

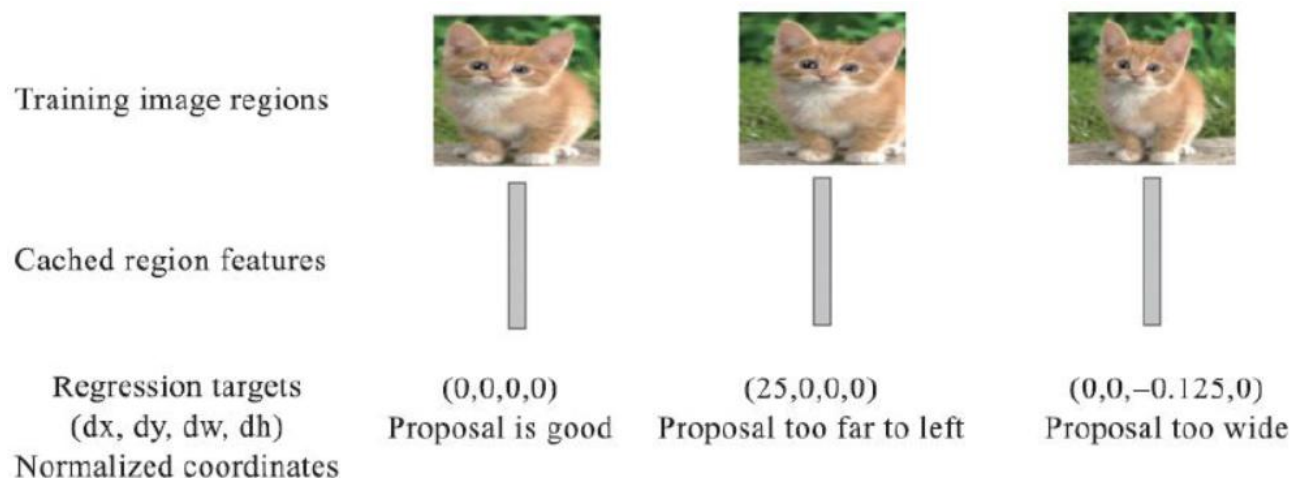


1) R-CNN

➤ 3. 训练最终的分类器

- 为每个类别单独训练一个SVM分类器。一个小技巧，SVM的训练也需要选择正负样本，R-CNN的提出者做了一个实验来选择最优IOU阈值，最终仅仅选择真实值的矩形框作为正样本

➤ 4. 训练回归模型：微调ROI位置



1) R-CNN

➤ 4. 训练回归模型：微调ROI位置

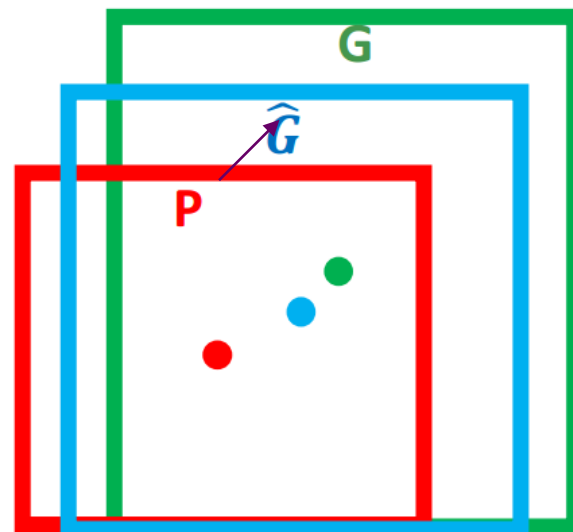
- 通过包围框回归(Bounding-box regression)提高定位准确率
- 具体目标
 - 窗口一般使用四维向量(x,y,w,h)来表示
 - 寻找一种关系使得输入原始的窗口P（红色）经过映射得到一个跟真实窗口G（绿色）更接近的回归窗口 $\hat{G}$ （蓝色）

$$(P_x, P_y, P_w, P_h) \xrightarrow{\quad} (G_x, G_y, G_w, G_h)$$

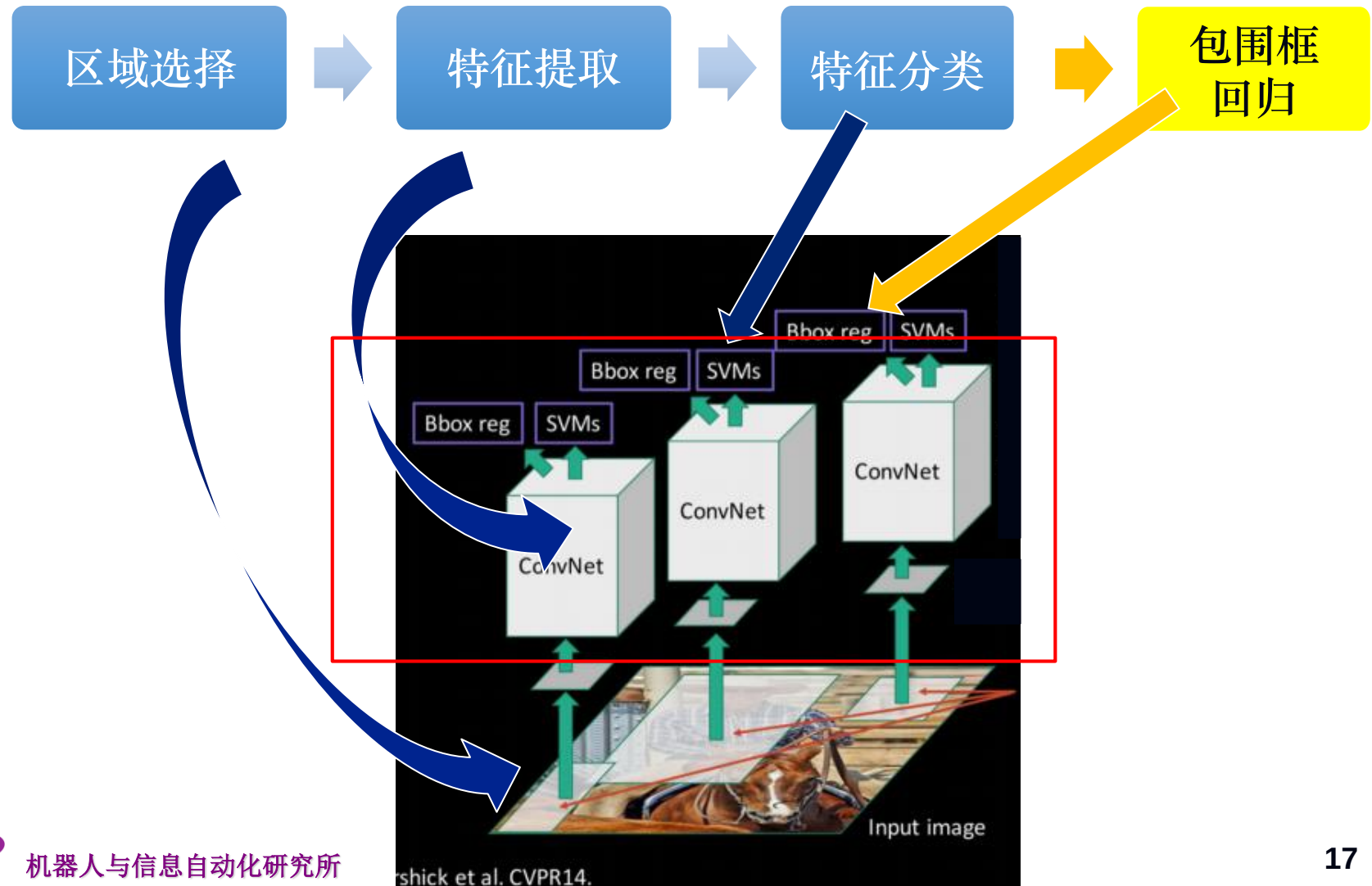


$$f(P_x, P_y, P_w, P_h) = (\hat{G}_x, \hat{G}_y, \hat{G}_w, \hat{G}_h)$$

$$(\hat{G}_x, \hat{G}_y, \hat{G}_w, \hat{G}_h) \approx (G_x, G_y, G_w, G_h)$$

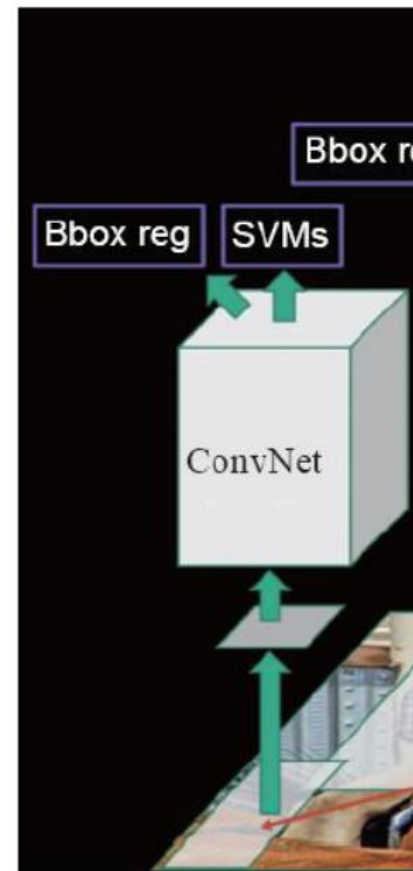


1) R-CNN



1) R-CNN

- R-CNN存在如下一些问题
 - 不论是训练还是预测，都需要对selective search出来的2000个ROI全部通过CNN的Forward过程来获取特征，这个过程花费的时间会非常长
 - 卷积神经网络的特征提取器和用来预测分类的SVM是分开的，也就是特征提取的过程不会因SVM和回归的调整而更新
 - R-CNN具有非常复杂的操作流程，而且每一步都是分裂的，如特征提取器通过Softmax分类获得，最终的分类结果由SVM获得，矩形框的位置则是通过回归方式获得



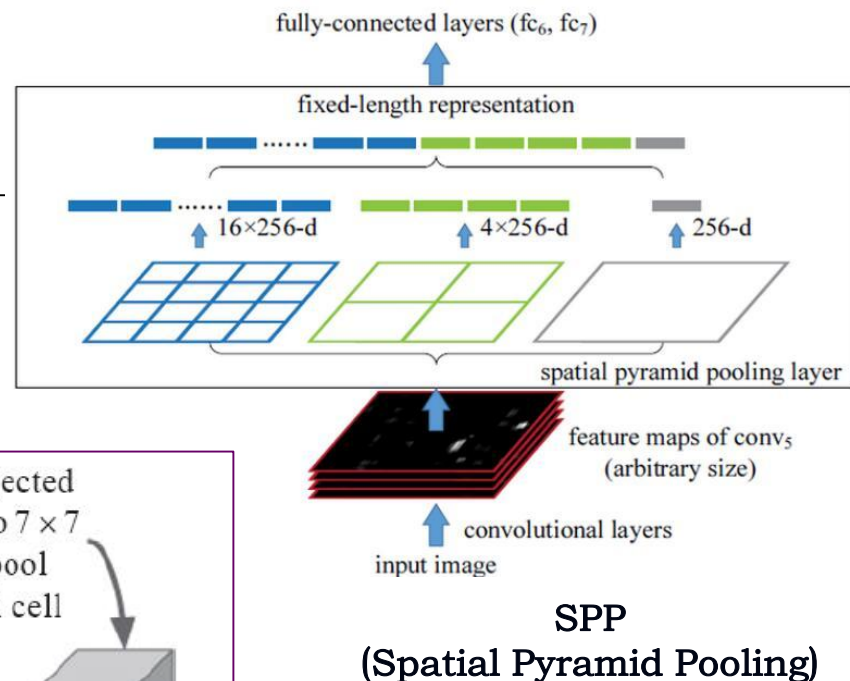
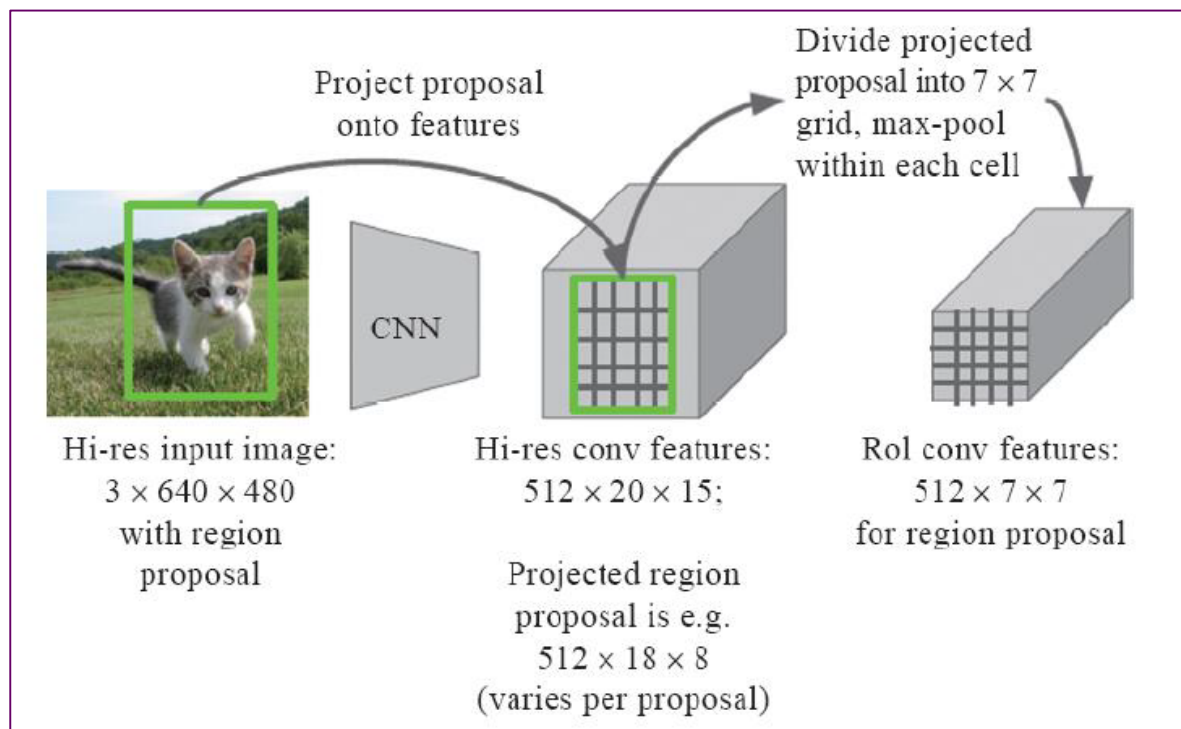
2) Fast R-CNN

➤ Fast R-CNN

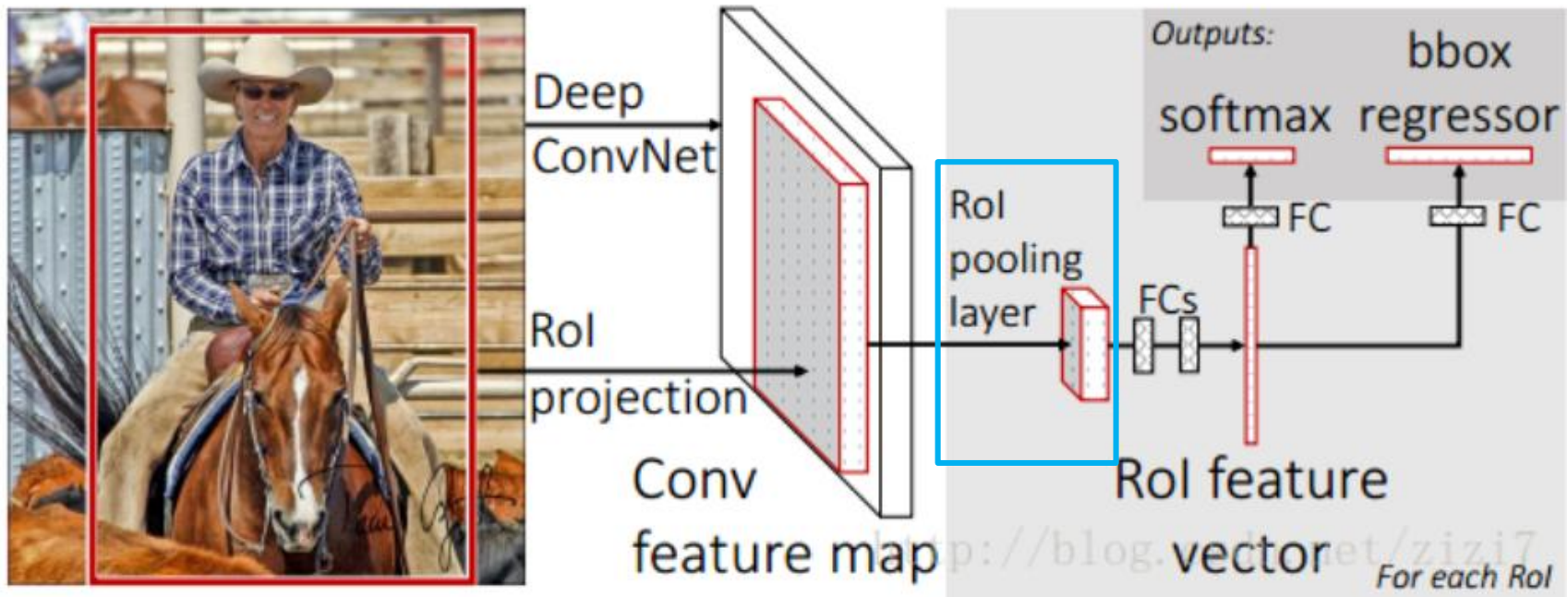
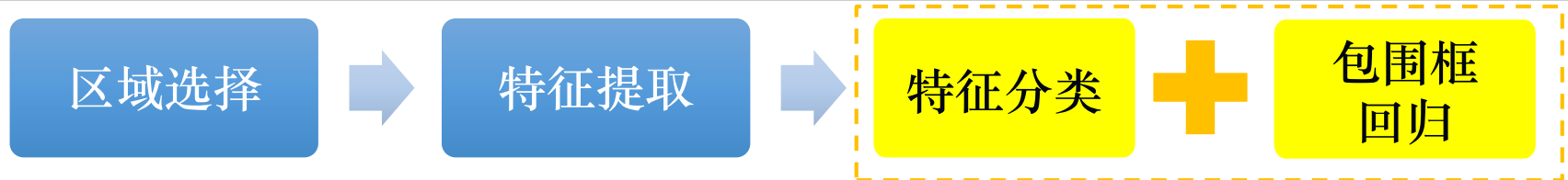
- 首先是速度，2000个ROI的CNN特征提取占用了大量的时间，是否可以使用更好的方法（比如，共享卷积层）来同时处理这2000个ROI？
- 其次是CNN的特征不会因SVM和回归的调整而更新。
- R-CNN的操作流程比较复杂，是否有更好的方式使得训练过程成为端到端的？

2) Fast R-CNN

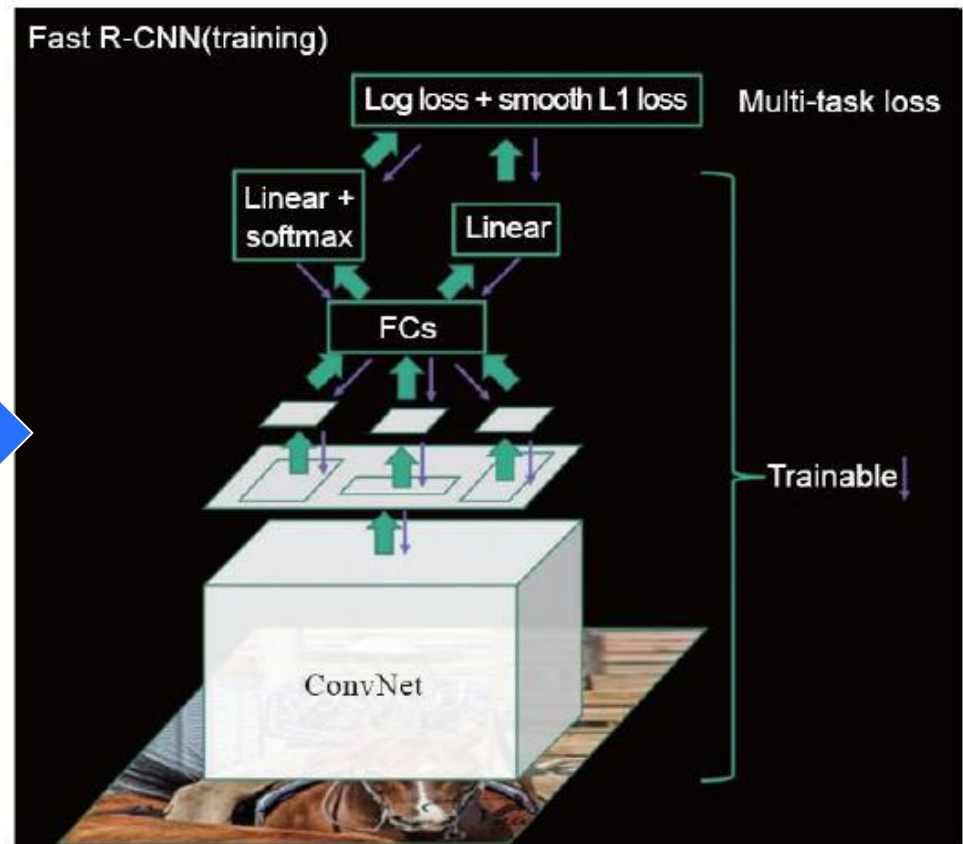
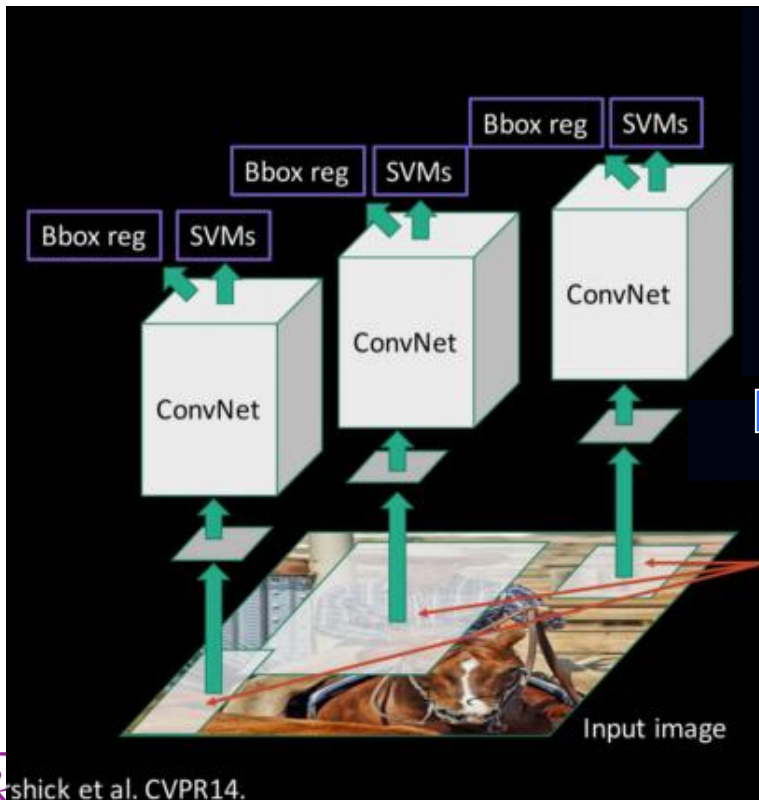
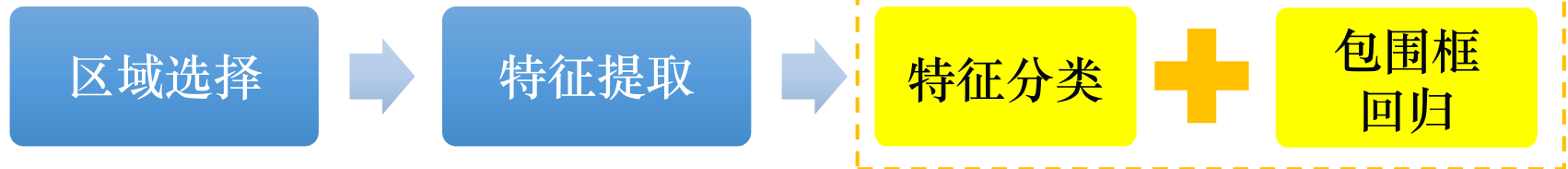
➤ ROI 池化



2) Fast R-CNN

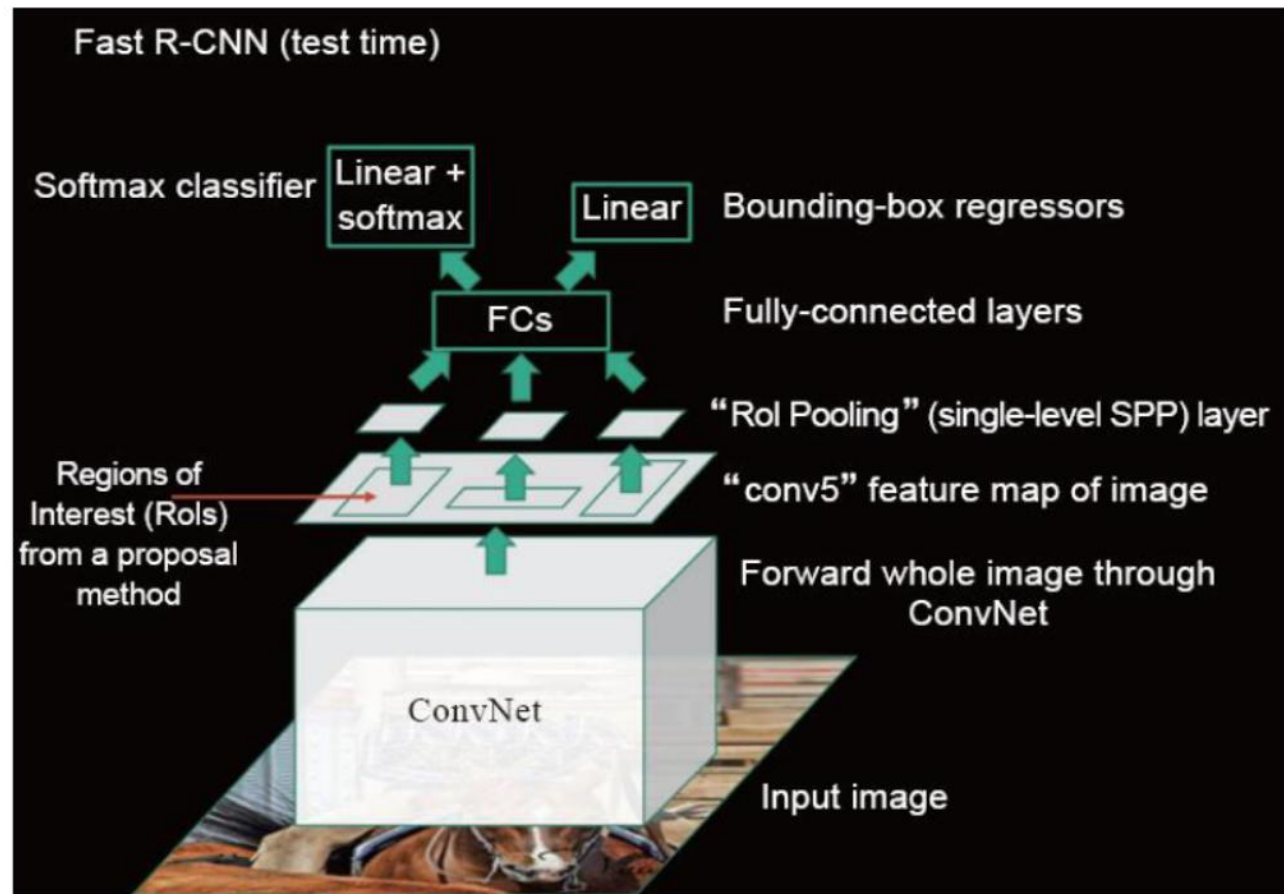


2) Fast R-CNN



2) Fast R-CNN

➤ Fast R-CNN预测过程

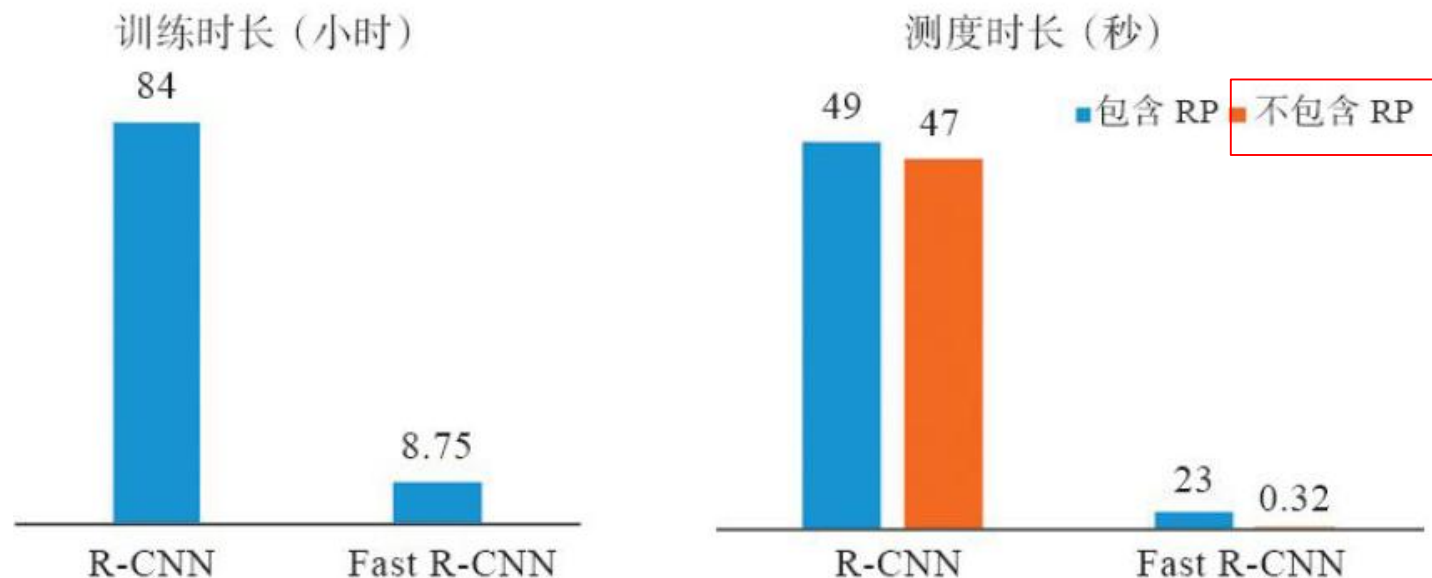


2) Fast R-CNN

➤ Fast R-CNN vs R-CNN

R-CNN: 许多候选框 -> CNN -> 得到每个候选框的特征 -> 分类+回归

Fast R-CNN: 一张图像 -> CNN -> 得到每张候选框的特征 -> 同时分类回归

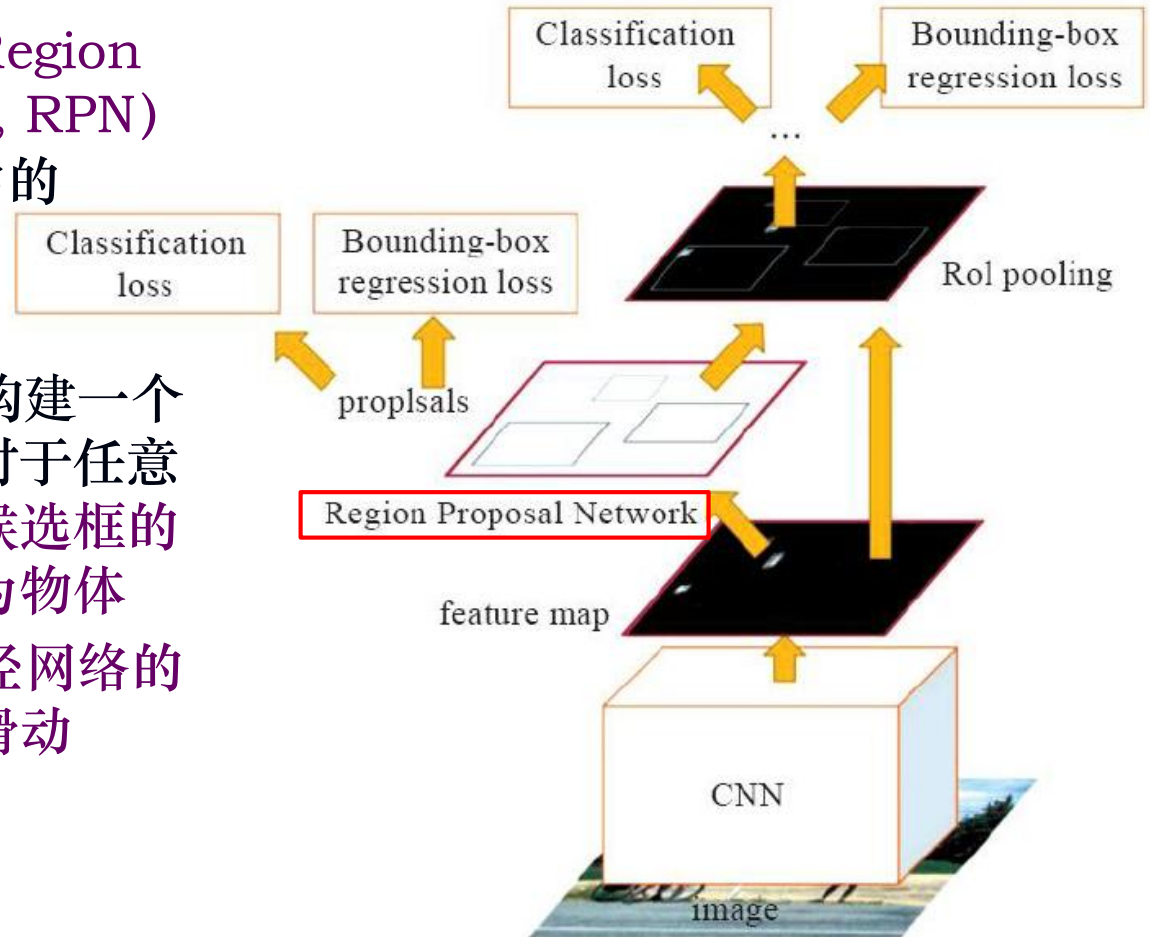


问题：处理候选框时间过长

3) Faster R-CNN

➤ Faster R-CNN算法思想

- 用区域候选网络 (Region proposal network, RPN) 代替Fast R-CNN中的selective search
- RPN的核心思想：构建一个小的全卷积网络，对于任意大小的图像，输出候选框的具体位置及其是否为物体
- RPN网络在卷积神经网络的最后一个特征层上滑动



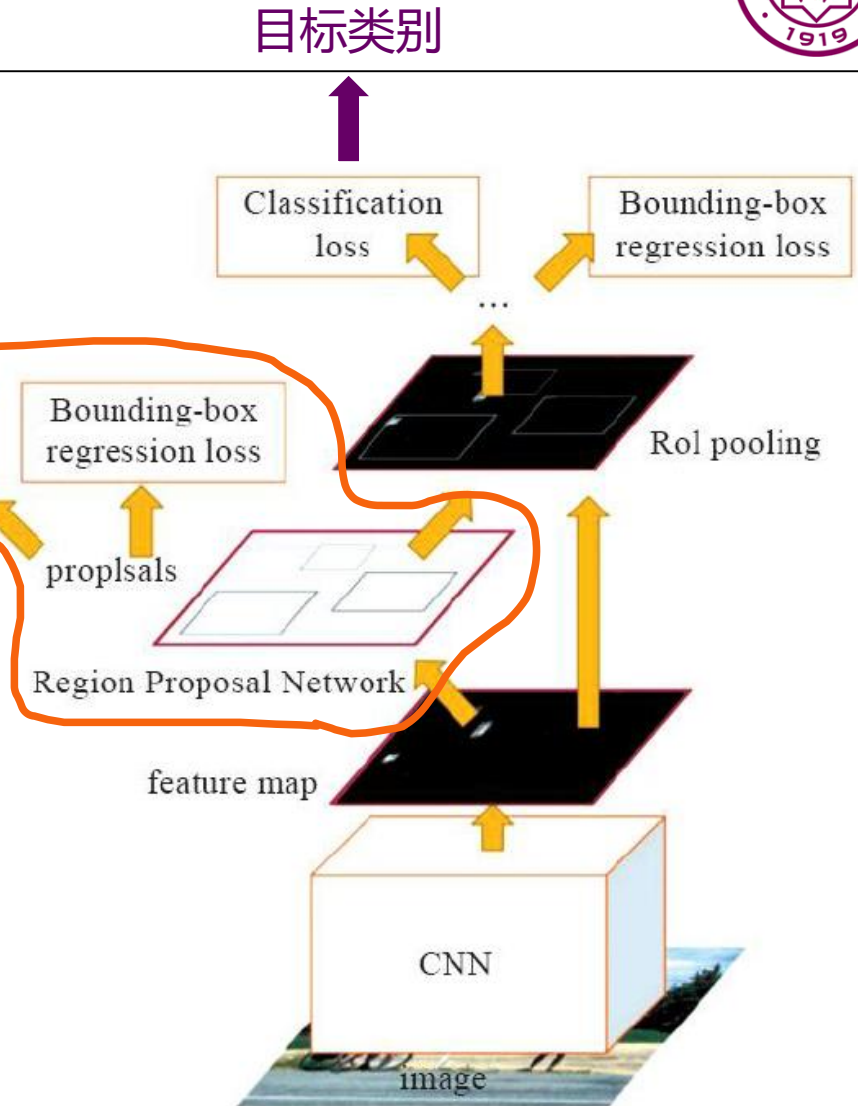
3) Faster R-CNN

➤ Faster R-CNN基本结构

- CNN: 从图像中提取特征图 (feature map), 特征图被 RPN 和全连接层共享

是否为候选区域 ←

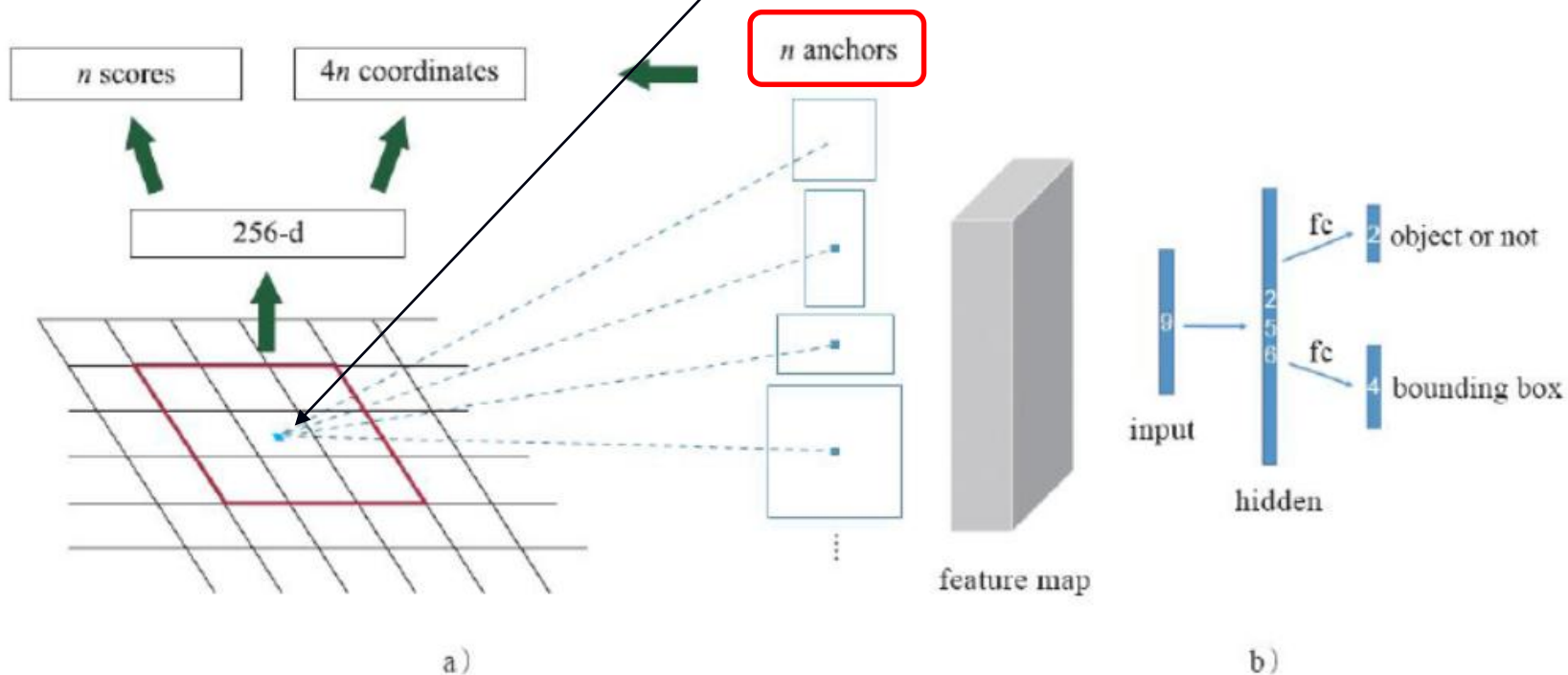
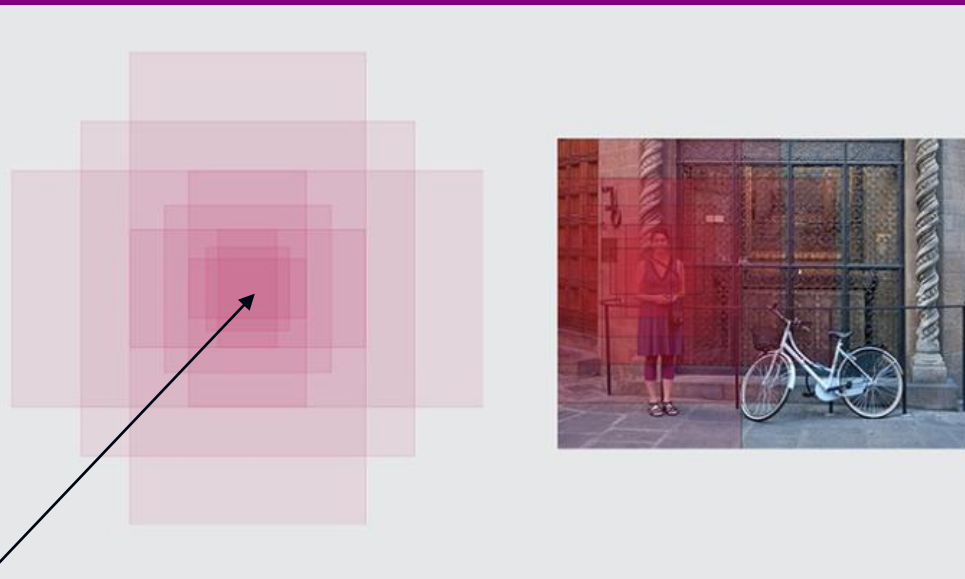
- RPN: 从anchor中产生候选框(proposals), **RPN中包含独立的分类和回归模块 (全连接层)**
- ROI pooling+全连接层: 生成候选框的图像特征, 判断目标类别并获得包围框的精确位置



3) Faster R-CNN

➤ RPN原理

- 核心思想: 构建一个小的片, 输出ROI的具体位置
络在卷积神经网的最后



3) Faster R-CNN

➤ RPN训练细节

- 针对分类任务，对于滑窗产生的每一个anchor都计算该anchor与真实标记矩形框的IOU。当IOU大于0.7时，认为该anchor中含有物体；当IOU小于0.3时，认为不包含物体；当IOU介于0.3~0.7时，则不参与网络训练的迭代过程。
- 对于回归任务，定义为anchor中心点的横、纵坐标以及anchor的宽、高，学习目标为anchor与真实bbox在这四个值上的偏移。RPN为一个全卷积网络，可以用随机梯度下降的方式端到端地进行训练。

3) Faster R-CNN

➤ Faster R-CNN的训练过程

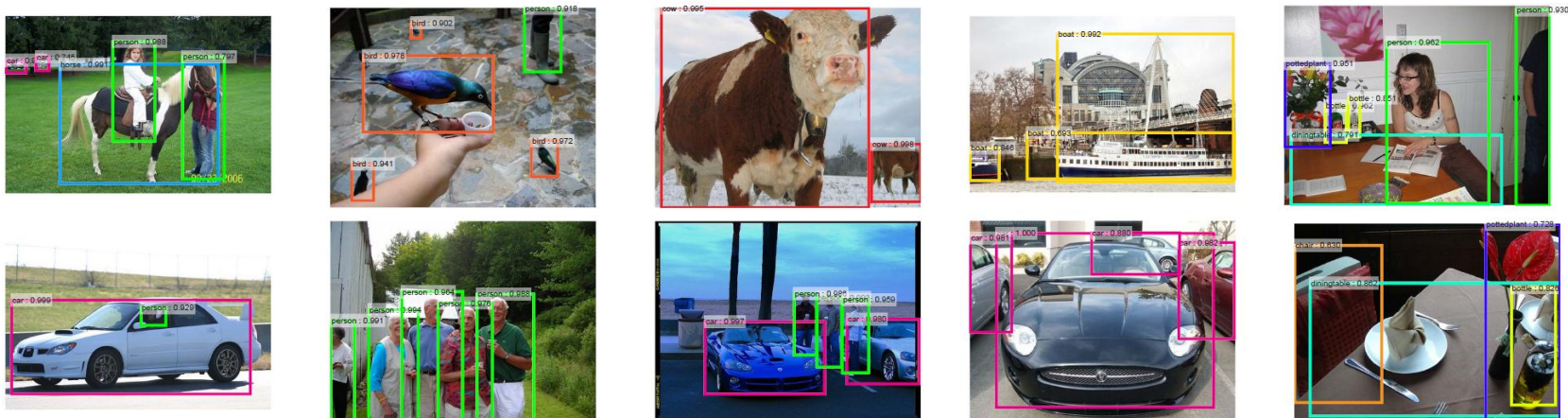
- 使用ImageNet预训练好的模型训练一个RPN
- 使用ImageNet预训练好的模型，以及第1步里产生的候选区域训练Faster R-CNN，得到物体的实际类别以及微调的矩形框位置
- 使用第2步中的网络初始化RPN，固定前面的卷积层，只调整RPN层的参数
- 固定前面的卷积层，只训练并调整Faster R-CNN的FC层

3) Faster R-CNN



Faster R-CNN预测阶段的具体步骤

1. 将图像输入网络得到对应的特征图feature maps
2. 使用RPN生成1000个候选框，将候选框投影到特征图上，获得相应的特征矩阵
3. 将每个特征矩阵通过ROI pooling缩放到7*7大小的特征图
4. 将特征图通过一系列全连接层，得到类别预测结果和针对每个类别的包围框回归结果

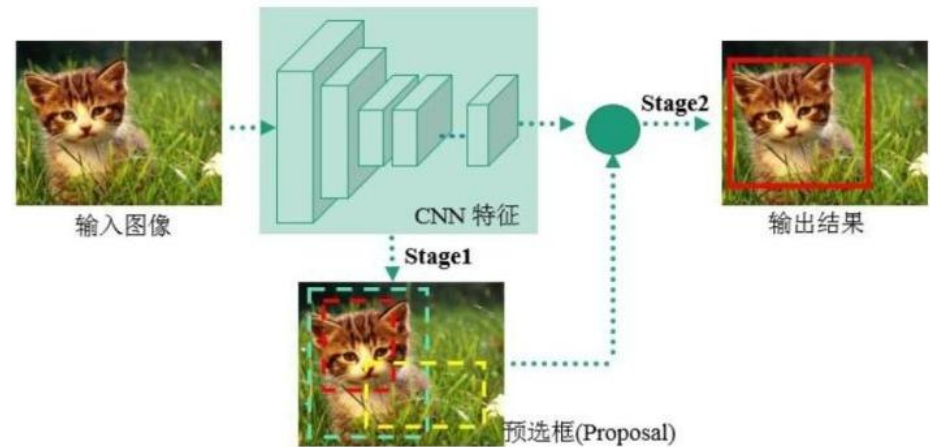
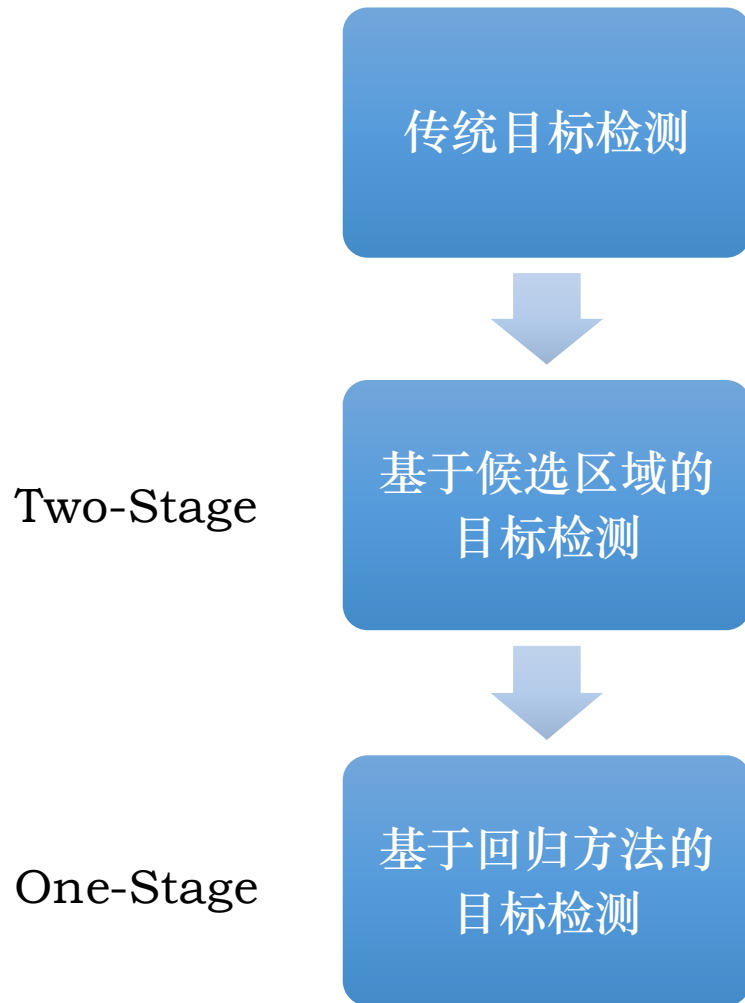


8.1.2 Two-stage目标检测方法

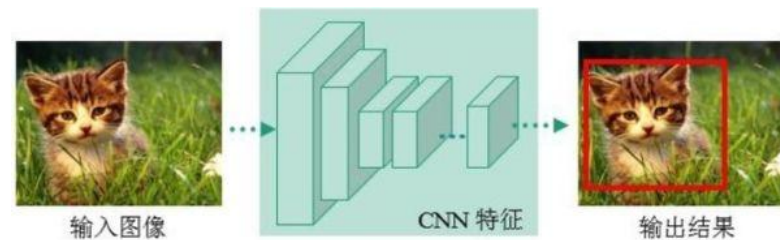
- 从R-CNN到Fast R-CNN，再到Faster R-CNN，目标检测的四个基本步骤（候选框生成、特征提取、分类、包围框回归）被逐渐统一到一个深度网络框架中
- Faster R-CNN的计算没有重复，可完全在GPU中完成，大大提高了运行速度
- 在简单网络（ZF net）目标检测速度达到17fps，在PASCAL VOC上准确率为59.9%
- 在复杂网络（VGG-16）目标检测速度达到5fps，准确率73.2%

预测时长：49s	2.3s	0.2s												
<table><tr><td colspan="2">region proplsal (SS)</td></tr><tr><td colspan="2">feature extraction (Deep Net)</td></tr><tr><td>classification (SVM)</td><td>rect refine (regression)</td></tr></table>	region proplsal (SS)		feature extraction (Deep Net)		classification (SVM)	rect refine (regression)	<table><tr><td colspan="2">region proplsal (SS)</td></tr><tr><td colspan="2">feature extraction classification + rect refine (Deep Net)</td></tr></table>	region proplsal (SS)		feature extraction classification + rect refine (Deep Net)		<table><tr><td colspan="2">region proposal feature extraction classification + rect refine (Deep Net)</td></tr></table>	region proposal feature extraction classification + rect refine (Deep Net)	
region proplsal (SS)														
feature extraction (Deep Net)														
classification (SVM)	rect refine (regression)													
region proplsal (SS)														
feature extraction classification + rect refine (Deep Net)														
region proposal feature extraction classification + rect refine (Deep Net)														
RCNN	fast RCNN	faster RCNN												

8.1 目标检测



知乎 @Hero



8.1.3 One-stage目标检测方法

You Only Look Once: Unified, Real-Time Object Detection

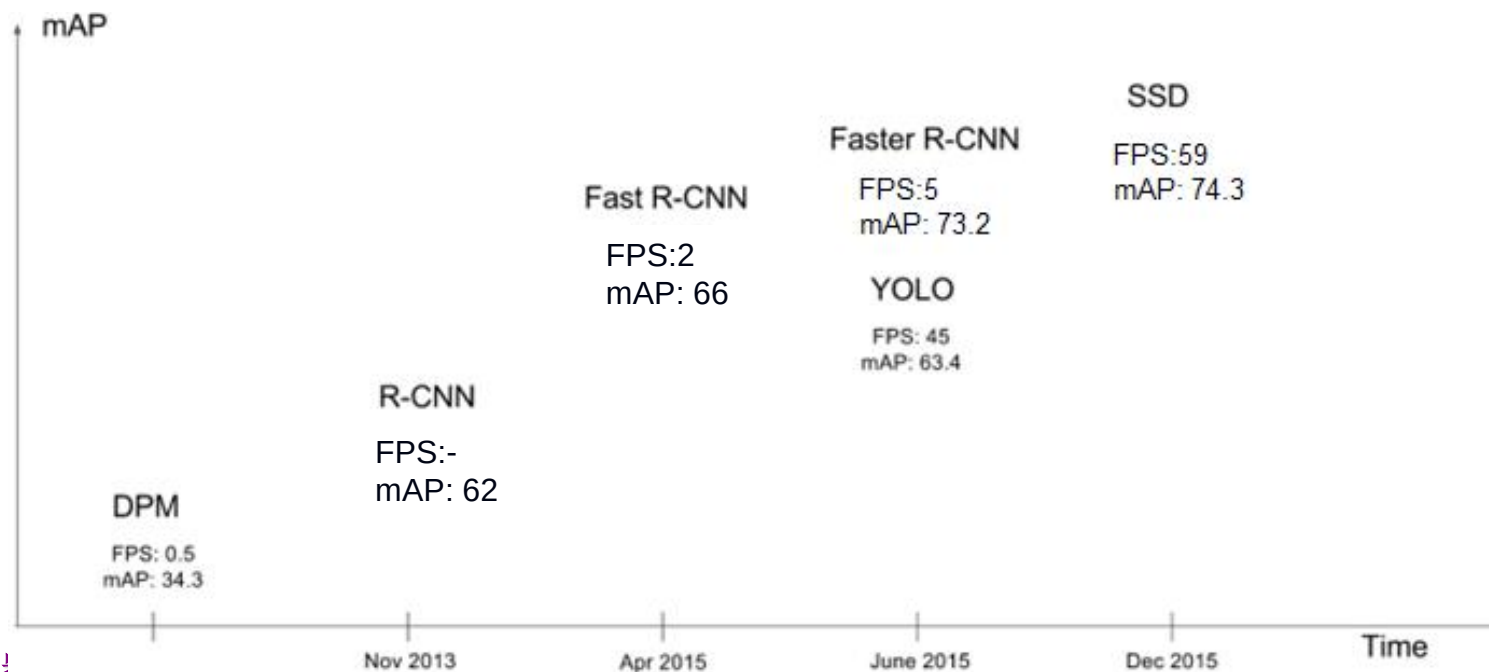
CVPR2016

Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi

SSD: Single Shot MultiBox Detector

ECCV2016

Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, etc.



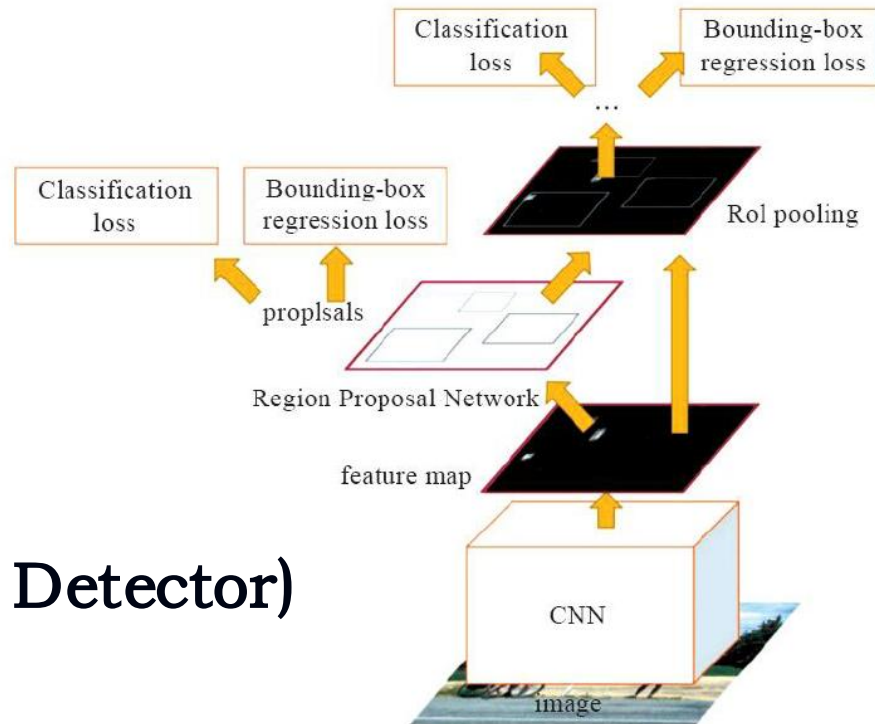
1) SSD

➤ Faster R-CNN的主要问题

- 对小目标检测效果差
- 检测速度较慢

➤ SSD(Single Shot MultiBox Detector)的主要特点

- 在**不同尺度的特征图**上进行检测，大尺度特征图用来检测小物体，小尺度特征图用来检测大物体
- 放弃RPN，直接基于不同尺度和长宽比的default bounding boxes/Anchors进行预测



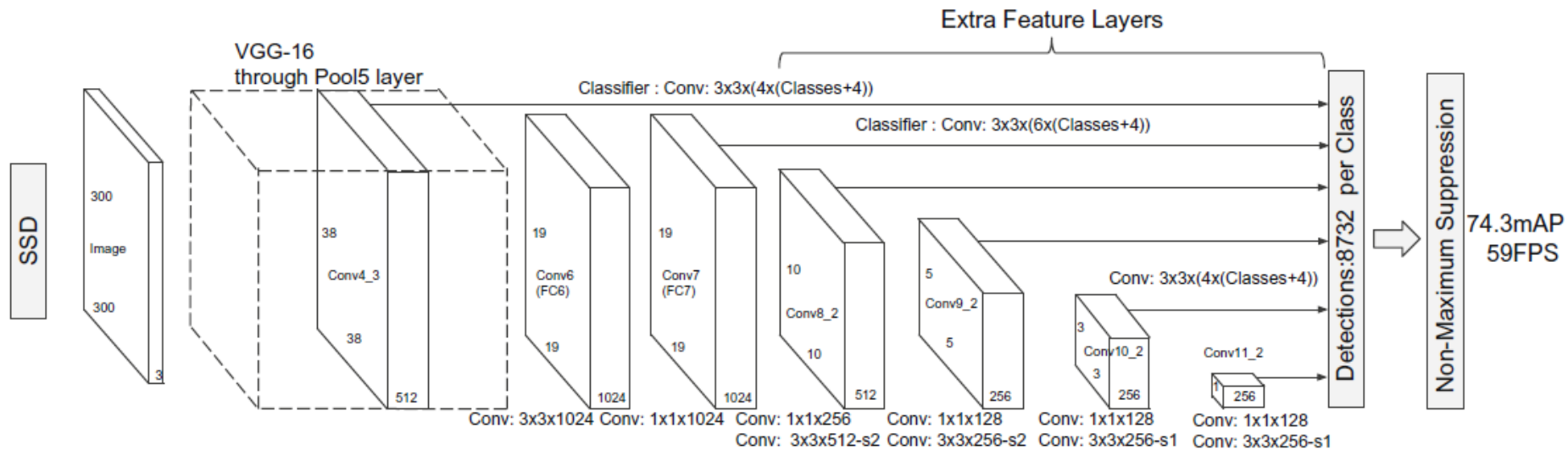
<https://arxiv.org/abs/1512.02325>

SSD: 网络结构

➤ 特征图

- 特征图6个: Conv4_3, Conv7, Conv8_2, Conv9_2, Conv10_2, Conv11_2层
- 特征图大小分别为: $38*38$, $19*19$, $10*10$, $5*5$, $3*3$, $1*1$

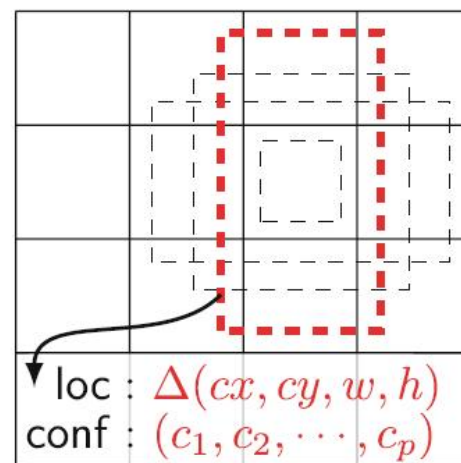
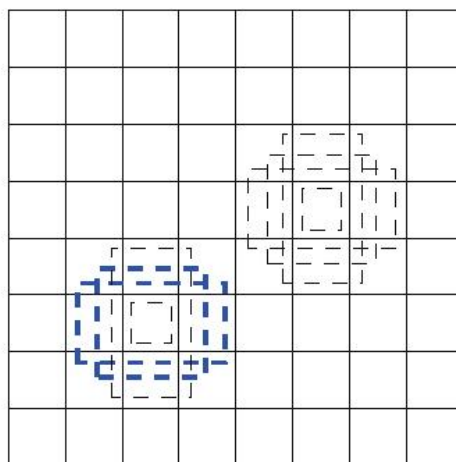
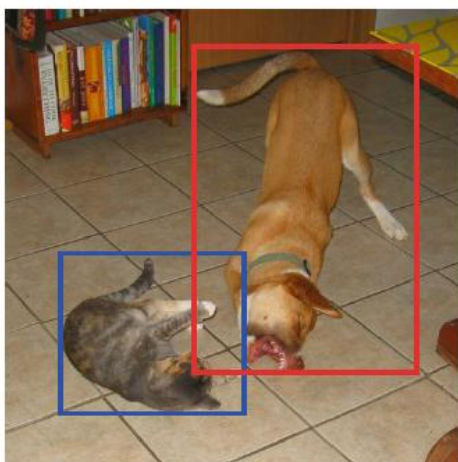
➤ 在不同特征尺度上预测不同尺度的目标



Default boxes/Anchors, 共8732个

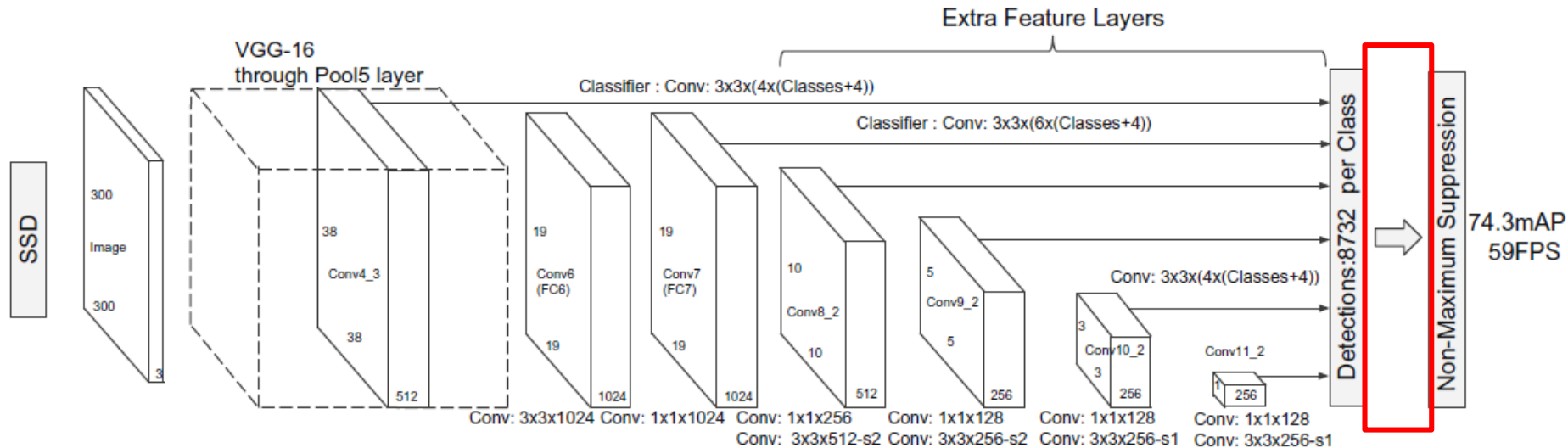
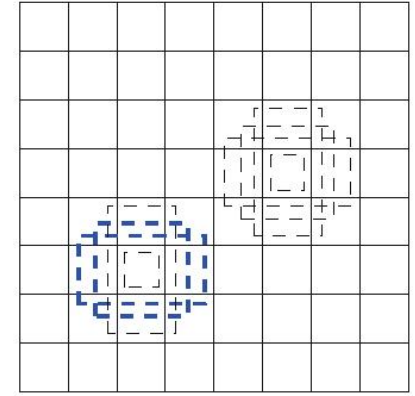


特征层	特征层尺寸	Anchor尺寸及比例	Anchor总数
Conv4_3	38*38	30{1/2, 1, 2}+{1}	38*38*4
Conv7	19*19	60{1/3, 1/2, 1, 2, 3}+{1}	19*19*6
Conv8_2	10*10	111{1/3, 1/2, 1, 2, 3}+{1}	10*10*6
Conv9_2	5*5	162{1/3, 1/2, 1, 2, 3}+{1}	5*5*6
Conv10_2	3*3	213{1/2, 1, 2}+{1}	3*3*4
Conv11_2	1*1	264{1/2, 1, 2}+{1}	1*1*4



SSD: 网络结构

- 使用3*3的卷积核完成最终预测
- 对于每个default box/anchor, 预测得到c个类别的得分, 以及4个包围框变换参数 (与faster R-CNN不同)
- 若特征图的每个位置有k个anchors, 则3*3的卷积核的深度为 $(c+4)*k$



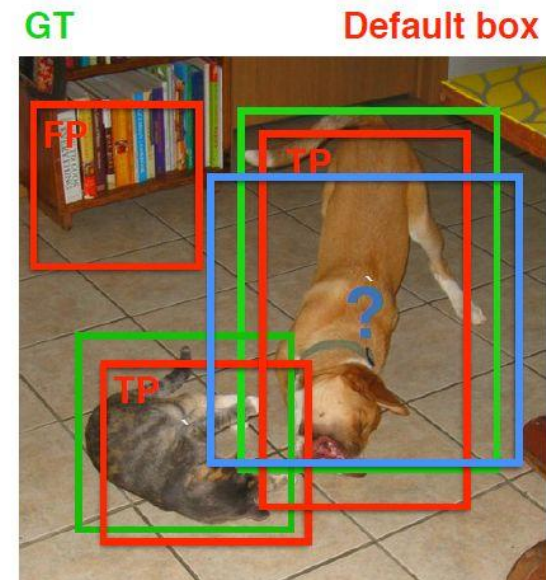
SSD: 正负样本选取

➤ 正样本选取

- 原则1: 对于图像中每个GT包围框, 找到与其IoU最大的 default box/anchor, 则该 default box 为正样本, 从而保证每个GT一定与某个 default box 匹配
- 原则2: 对于剩余未匹配的 default box, 若其与某个GT包围框的IoU大于阈值 (一般是0.5), 则该 default box 也为正样本, 每个 default box 只能与一个GT匹配

➤ 举例

	DB1	DB2	DB3
GT1	0.4	0.2	0.3
GT2	0.6	0.8	0.7



SSD: 正负样本选取

➤ 正样本选取

- 原则1: 对于图像中每个GT包围框, 找到与其IoU最大的 default box/anchor, 则该 default box 为正样本, 从而保证每个GT一定与某个 default box 匹配
- 原则2: 对于剩余未匹配的 default box, 若其与某个GT包围框的IoU大于阈值 (一般是0.5), 则该 default box 也为正样本, 每个 default box 只能与一个GT匹配

➤ 难例挖掘(hard negative mining)

- 对负样本根据 **confidence loss** (置信度) 进行降序排列
- 选取置信度最大的 top-k 个 default box 作为训练的负样本, 以保证正负样本比例接近 1:3

SSD：预测步骤



SSD预测阶段的具体步骤

1. 将图像输入SSD网络，对于图像中的每个default box，得到预测的目标类别概率（置信度）、包围框信息
2. 根据类别置信度确定其类别，去掉属于背景的预测包围框
3. 根据置信度阈值过滤掉阈值较低的预测包围框
4. 对于留下的预测包围框，根据其对应的default box计算其真实的位置/尺寸参数（解码）
5. 根据预测包围框的类别置信度进行降序排列，然后仅保留top-k个预测包围框
6. 使用NMS算法过滤掉重叠度较大的预测包围框，最后剩余就是检测结果

补充：非最大抑制（NMS）

Algorithm 1 Non-Maximum Suppression Algorithm

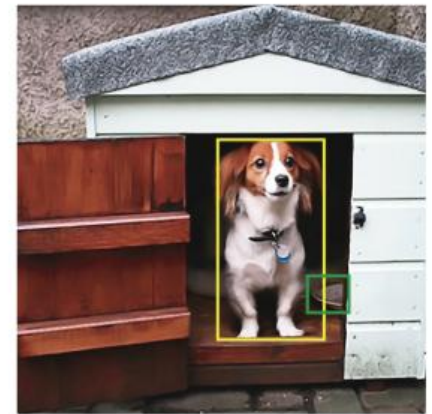
Require: Set of predicted bounding boxes B , confidence scores S , IoU threshold τ , confidence threshold T

Ensure: Set of filtered bounding boxes F

```
1:  $F \leftarrow \emptyset$ 
2: Filter the boxes:  $B \leftarrow \{b \in B \mid S(b) \geq T\}$ 
3: Sort the boxes  $B$  by their confidence scores in descending order
4: while  $B \neq \emptyset$  do
5:   Select the box  $b$  with the highest confidence score
6:   Add  $b$  to the set of final boxes  $F$ :  $F \leftarrow F \cup \{b\}$ 
7:   Remove  $b$  from the set of boxes  $B$ :  $B \leftarrow B - \{b\}$ 
8:   for all remaining boxes  $r$  in  $B$  do
9:     Calculate the IoU between  $b$  and  $r$ :  $iou \leftarrow IoU(b, r)$ 
10:    if  $iou \geq \tau$  then
11:      Remove  $r$  from the set of boxes  $B$ :  $B \leftarrow B - \{r\}$ 
12:    end if
13:  end for
14: end while
```



➡
NMS



SSD: 实验结果



VOC2007

Method	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
Fast [6]	70.0	77.0	78.1	69.3	59.4	38.3	81.6	78.6	86.7	42.8	78.8	68.9	84.7	82.0	76.6	69.9	31.8	70.1	74.8	80.4	70.4
Faster [2]	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
SSD300	72.1	75.2	79.8	70.5	62.5	41.3	81.1	80.8	86.4	51.5	74.3	72.3	83.5	84.6	80.6	74.5	46.0	71.4	73.8	83.0	69.1
SSD500	75.1	79.8	79.5	74.5	63.4	51.9	84.9	85.6	87.2	56.6	80.1	70.0	85.4	84.9	80.9	78.2	49.0	78.4	72.4	84.6	75.5

	SSD300					
more data augmentation?		✓	✓	✓	✓	✓
use conv4_3?	✓		✓	✓	✓	✓
include $\{\frac{1}{2}, 2\}$ box?	✓	✓		✓	✓	✓
include $\{\frac{1}{3}, 3\}$ box?	✓	✓			✓	✓
use atrous?	✓	✓	✓	✓		✓
VOC2007 test mAP	65.4	68.1	69.2	71.2	71.4	72.1

SSD: 总结

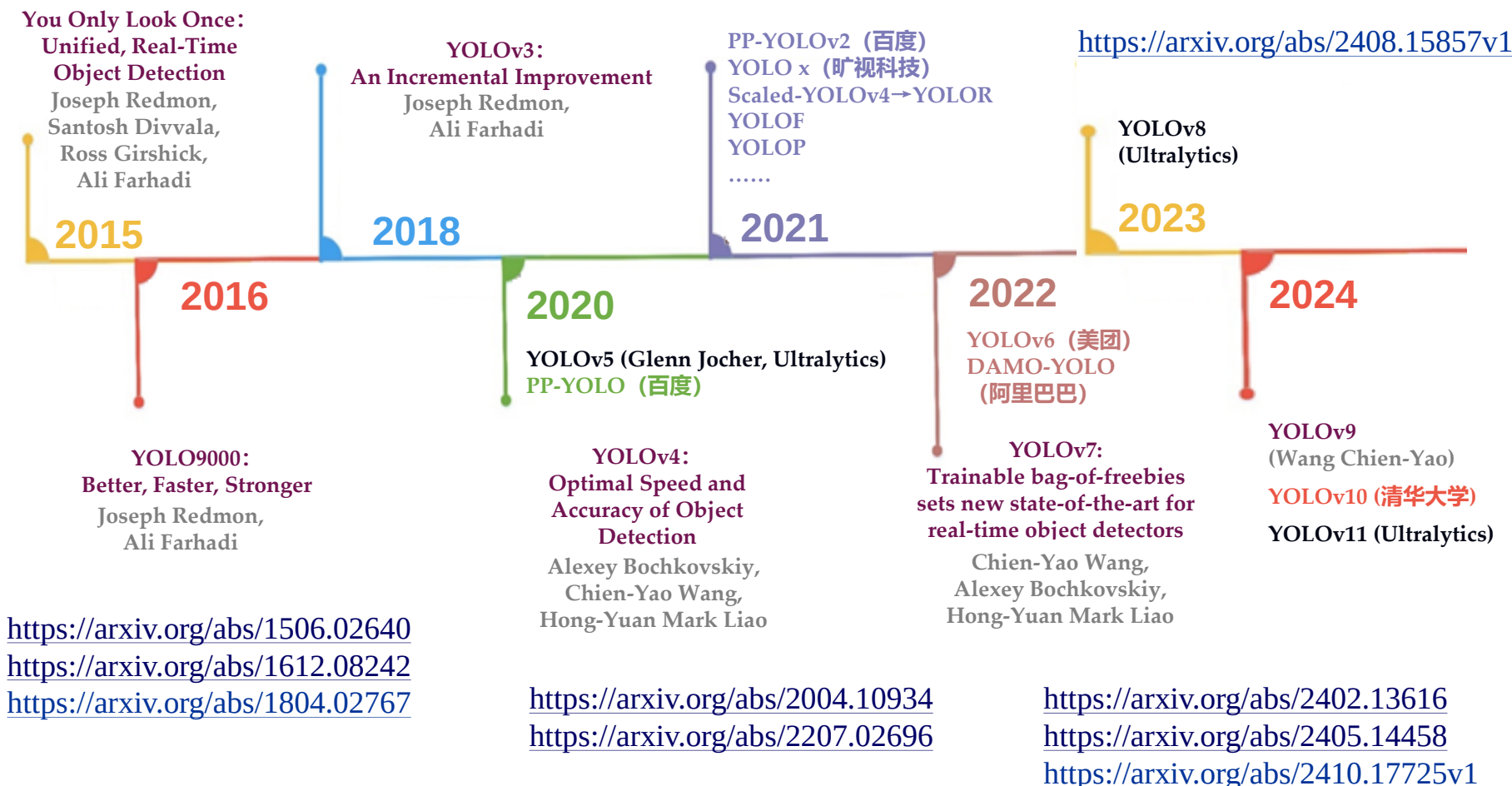
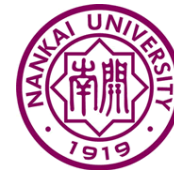


➤ 效果

- 准确率：由于使用多个不同尺度的特征图进行预测，再加上多个不同长宽比的default box，检测准确率相较于YOLO有很大提升，达到了Faster R-CNN的效果
- 运行速度：检测速度由YOLO的45fps提到高了58fps



2) YOLO系列



YOLO v3

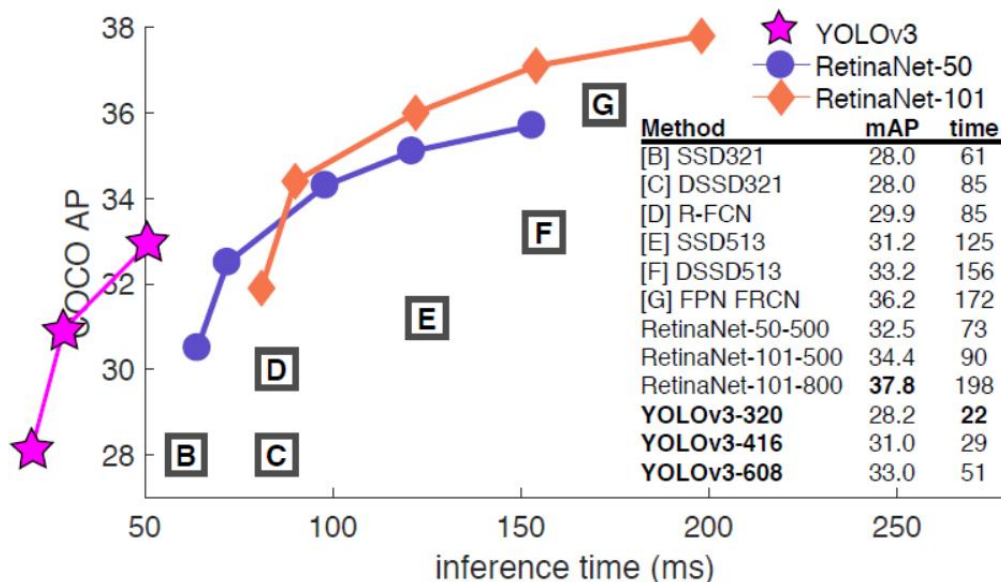


Figure 1. We adapt this figure from the Focal Loss paper [9]. YOLOv3 runs significantly faster than other detection methods with comparable performance. Times from either an M40 or Titan X, they are basically the same GPU.

[9] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*, 2017. 1, 3, 4



Abstract

We present some updates to YOLO! We made a bunch of little design changes to make it better. We also trained this new network that's pretty swell. It's a little bigger than last time but more accurate. It's still fast though, don't worry. At 320×320 YOLOv3 runs in 22 ms at 28.2 mAP, as accurate as SSD but three times faster. When we look at the old .5 IOU mAP detection metric YOLOv3 is quite good. It achieves 57.9 AP₅₀ in 51 ms on a Titan X, compared to 57.5 AP₅₀ in 198 ms by RetinaNet, similar performance but 3.8× faster. As always, all the code is online at <https://pjreddie.com/yolo/>.

1. Introduction

Sometimes you just kinda phone it in for a year, you know? I didn't do a whole lot of research this year. Spent a lot of time on Twitter. Played around with GANs a little. I had a little momentum left over from last year [12] [1]; I managed to make some improvements to YOLO. But, honestly, nothing like super interesting, just a bunch of small changes that make it better. I also helped out with other people's research a little.

YOLO v3: 网络结构

	Type	Filters	Size	Output
1x	Convolutional	32	3×3	256×256
	Convolutional	64	$3 \times 3 / 2$	128×128
	Convolutional	32	1×1	128×128
	Convolutional	64	3×3	
	Residual			
2x	Convolutional	128	$3 \times 3 / 2$	64×64
	Convolutional	64	1×1	64×64
	Convolutional	128	3×3	
	Residual			
	Convolutional	256	$3 \times 3 / 2$	32×32
8x	Convolutional	128	1×1	32×32
	Convolutional	256	3×3	
	Residual			
	Convolutional	512	$3 \times 3 / 2$	16×16
	Convolutional	256	1×1	16×16
8x	Convolutional	512	3×3	
	Residual			
	Convolutional	1024	$3 \times 3 / 2$	8×8
	Convolutional	512	1×1	8×8
	Convolutional	1024	3×3	
4x	Residual			
	Avgpool		Global	
	Connected		1000	
	Softmax			

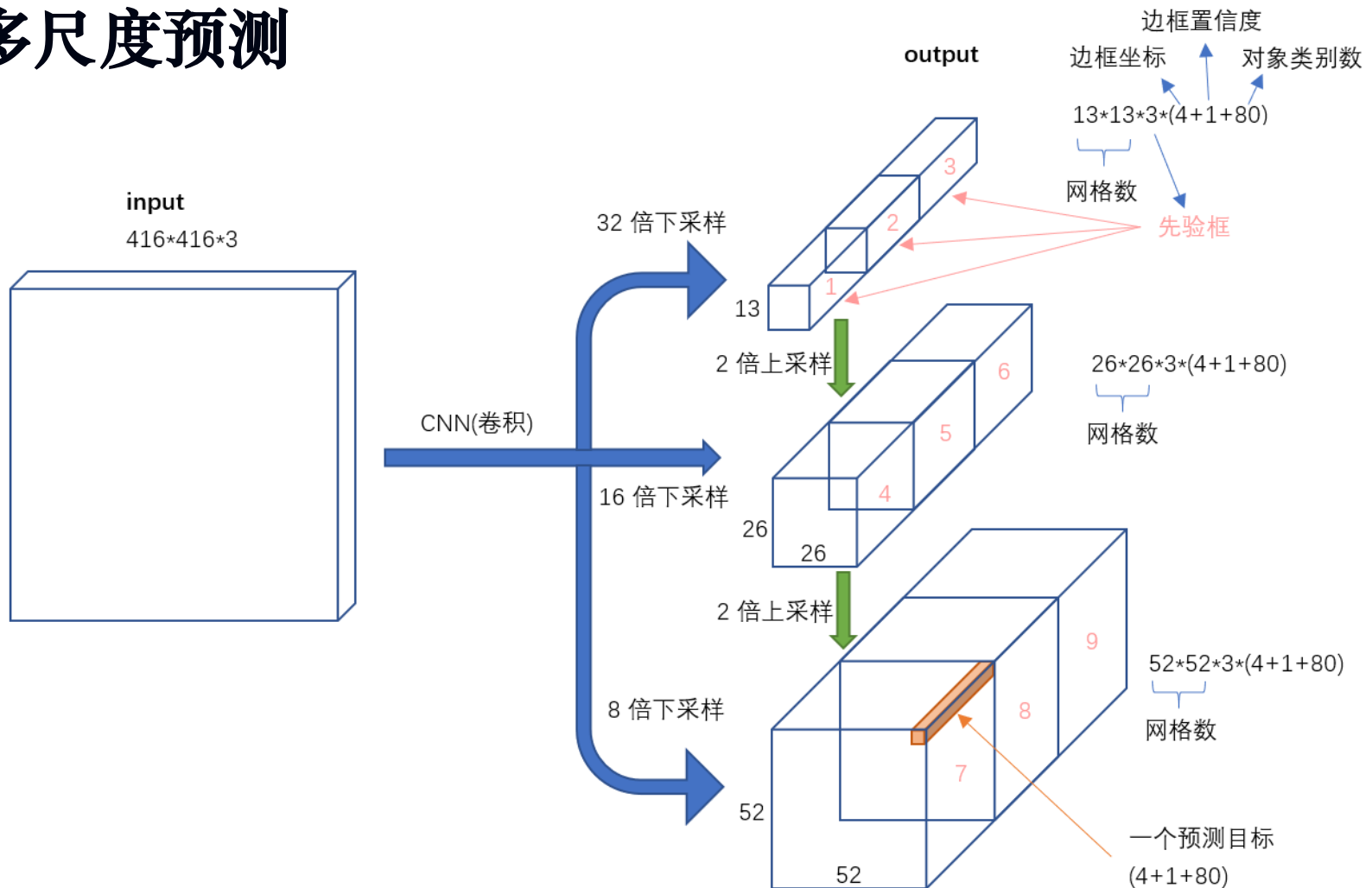
Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [15]	74.1	91.8	7.29	1246	171
ResNet-101[5]	77.1	93.7	19.7	1039	53
ResNet-152 [5]	77.6	93.8	29.4	1090	37
Darknet-53	77.2	93.8	18.7	1457	78

	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

Darknet-53

YOLO v3: 网络结构

➤ 多尺度预测



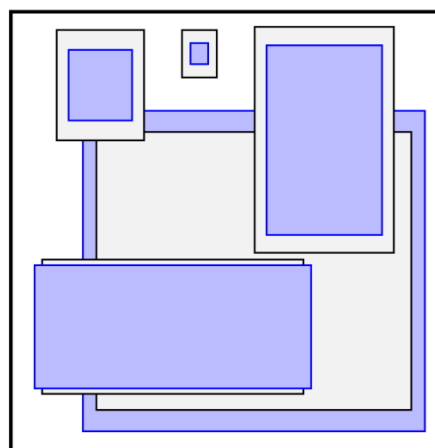
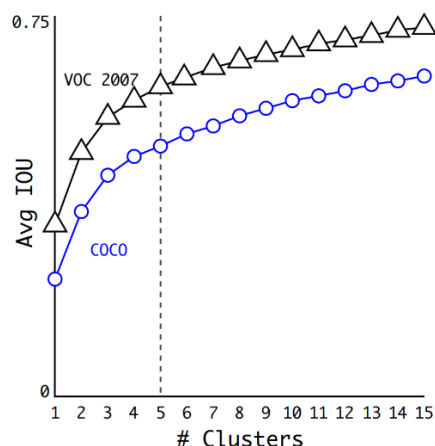
YOLO v3: Anchor/ Bounding box prior



- 通过对训练集中标注的GT包围框进行K-means聚类，统计出更符合样本中对象尺寸的anchor宽高
- 聚类中，计算包围框距离的公式：

$$d(\text{box}, \text{centroid}) = 1 - \text{IOU}(\text{box}, \text{centroid})$$

- centroid: 聚类时被选为中心的包围框
- box: 其他包围框



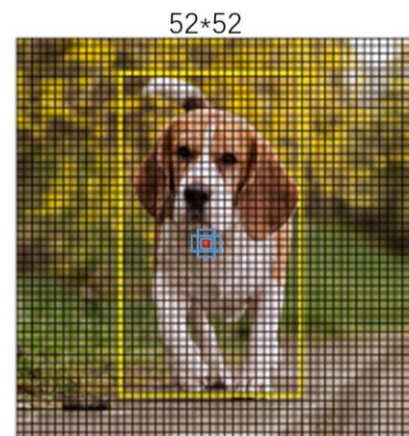
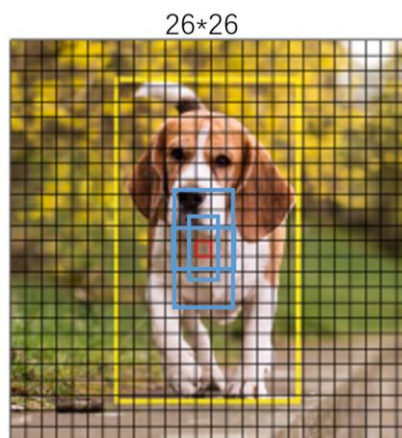
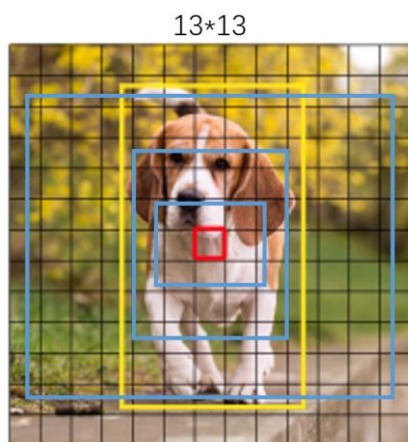
YOLO v2 聚类结果

YOLO v3: Anchor

- 通过对训练集中标注的GT包围框进行K-means聚类，统计出更符合样本中对象尺寸的anchor宽高

COCO数据集的聚类结果

特征层尺寸	Anchor尺寸	Anchor总数
13*13	116*90, 156*198, 373 *326	13*13*3
26*26	30*61, 62*45, 59*119	26*26*3
52*52	10*13, 16*30, 33*23	52*52*3



YOLO v3: 包围框映射

- 将预测包围框的**中心**限制在当前的grid网格中

$$x = (t_x * w_a) + x_a$$

$$y = (t_y * h_a) + y_a$$



预测
包围框
位置
尺寸

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

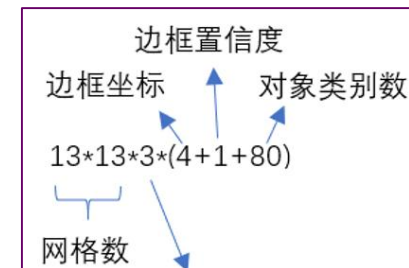
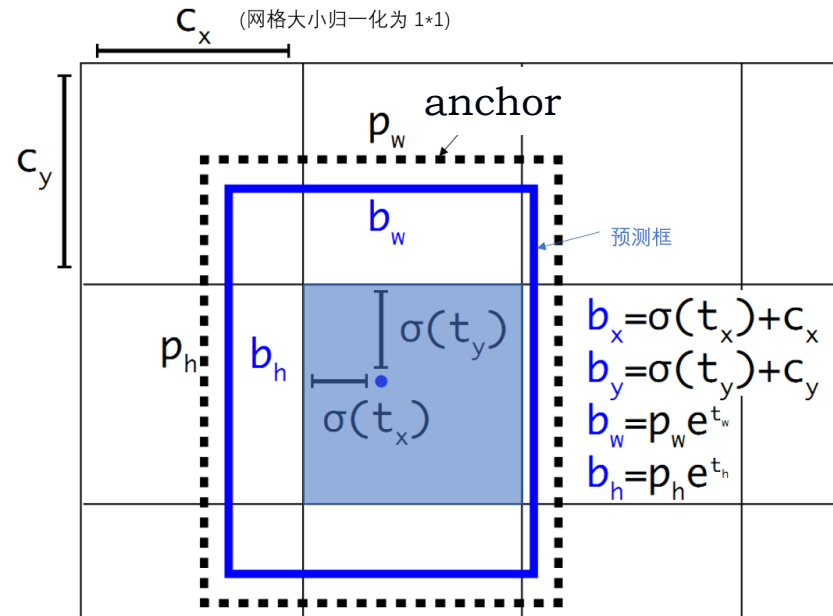
$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

预测包围框
置信度

$$Pr(\text{object}) * IOU(b, \text{object}) = \sigma(t_o)$$

$$\sigma(x) = \text{Sigmoid}(x)$$

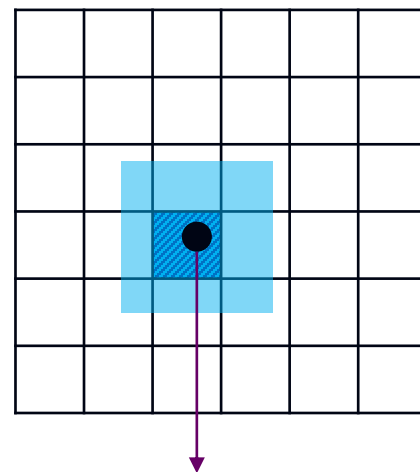


YOLO v3: 正负样本与损失函数



➤ 正负样本选取

- GT包围框的中心点落在特征层的一个Grid网格中，则该网格的Anchor负责检测该物体
- 将和GT重合最大IOU的anchor作为正样本（每个GT只分配一个正样本）
- 将和GT重合且IOU超过设定阈值的anchor丢弃
- 剩下的anchor为作为负样本



GT包围框中心

➤ 损失函数

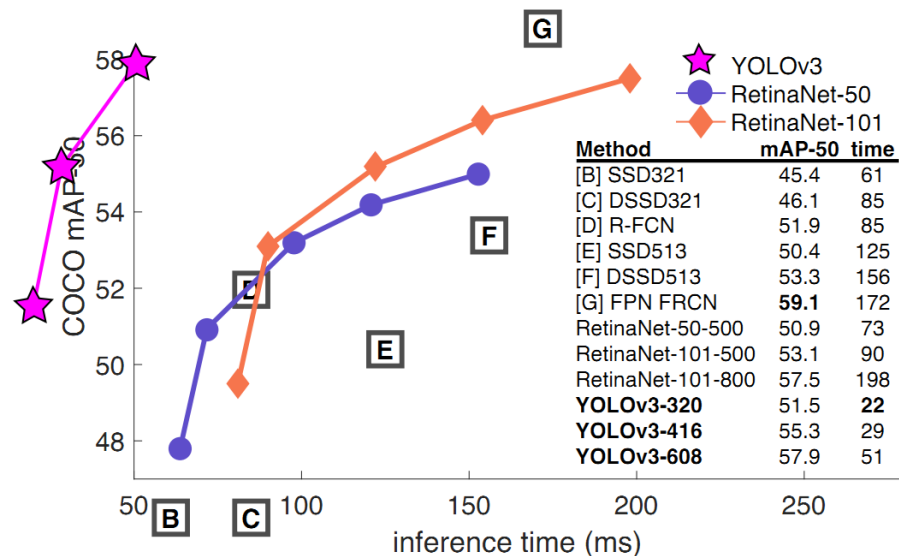
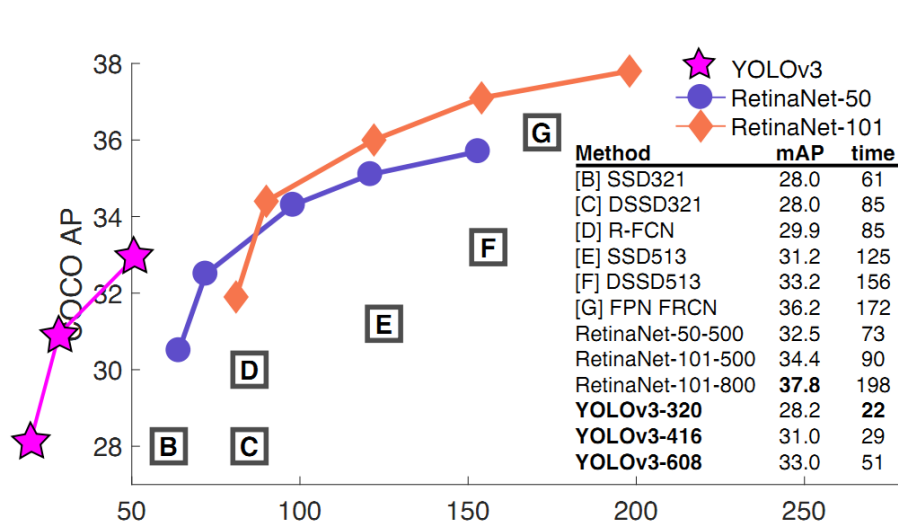
$$L = \lambda_{loc} L_{loc}(t, g) + \lambda_{cls} L_{cls}(O, C) + \lambda_{conf} L_{conf}(o, c)$$

定位损失
(L_2 loss)

分类损失
(BCE loss)

置信度损失
(BCE loss)

YOLO v3: 实验结果



	backbone	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
<i>Two-stage methods</i>							
Faster R-CNN+++ [5]	ResNet-101-C4	34.9	55.7	37.4	15.6	38.7	50.9
Faster R-CNN w FPN [8]	ResNet-101-FPN	36.2	59.1	39.0	18.2	39.0	48.2
Faster R-CNN by G-RMI [6]	Inception-ResNet-v2 [21]	34.7	55.5	36.7	13.5	38.1	52.0
Faster R-CNN w TDM [20]	Inception-ResNet-v2-TDM	36.8	57.7	39.2	16.2	39.8	52.1
<i>One-stage methods</i>							
YOLOv2 [15]	DarkNet-19 [15]	21.6	44.0	19.2	5.0	22.4	35.5
SSD513 [11, 3]	ResNet-101-SSD	31.2	50.4	33.3	10.2	34.5	49.8
DSSD513 [3]	ResNet-101-DSSD	33.2	53.3	35.2	13.0	35.4	51.1
RetinaNet [9]	ResNet-101-FPN	39.1	59.1	42.3	21.8	42.7	50.2
RetinaNet [9]	ResNeXt-101-FPN	40.8	61.1	44.1	24.1	44.2	51.2
YOLOv3 608 × 608	Darknet-53	33.0	57.9	34.4	18.3	35.4	41.9

2) YOLO系列



Release	Year	Tasks	Contributions	Framework
YOLO [5]	2015	Object Detection, Basic Classification	Single-stage object detector	Darknet
YOLOv2 [7]	2016	Object Detection, Improved Classification	Multi-scale training, dimension clustering	Darknet
YOLOv3 [8]	2018	Object Detection, Multi-scale Detection	SPP block, Darknet-53 backbone	Darknet
YOLOv4 [9]	2020	Object Detection, Basic Object Tracking	Mish activation, CSPDarknet-53 backbone	Darknet
YOLOv5 [10]	2020	Object Detection, Basic Instance Segmentation (via custom modifications)	Anchor-free detection, SWISH activation, PANet	PyTorch
YOLOv6 [11]	2022	Object Detection, Instance Segmentation	Self-attention, anchor-free OD	PyTorch
YOLOv7 [12]	2022	Object Detection, Object Tracking, Instance Segmentation	Transformers, E-ELAN reparameterisation	PyTorch
YOLOv8 [13]	2023	Object Detection, Instance Segmentation, Panoptic Segmentation, Key-point Estimation	GANs, anchor-free detection	PyTorch
YOLOv9 [14]	2024	Object Detection, Instance Segmentation	PGI and GELAN	PyTorch
YOLOv10 [15]	2024	Object Detection	Consistent dual assignments for NMS-free training	PyTorch

YOLO v11

<https://arxiv.org/abs/2410.17725v1>



YOLO v5

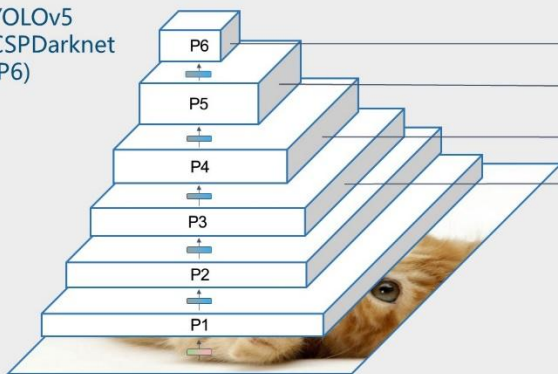


Model	size (pixels)	mAP ^{val} 50-95	mAP ^{val} 50	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5n	640	28.0	45.7	45	6.3	0.6	1.9	4.5
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5
YOLOv5m	640	45.4	64.1	224	8.2	1.7	21.2	49.0
YOLOv5l	640	49.0	67.3	430	10.1	2.7	46.5	109.1
YOLOv5x	640	50.7	68.9	766	12.1	4.8	86.7	205.7
YOLOv5n6	1280	36.0	54.4	153	8.1	2.1	3.2	4.6
YOLOv5s6	1280	44.8	63.7	385	8.2	3.6	12.6	16.8
YOLOv5m6	1280	51.3	69.3	887	11.1	6.8	35.7	50.0
YOLOv5l6	1280	53.7	71.3	1784	15.8	10.5	76.8	111.4
YOLOv5x6	1280	55.0	72.7	3136	26.2	19.4	140.7	209.8
+ TTA	1536	55.8	72.7	-	-	-	-	-

YOLOv5-I

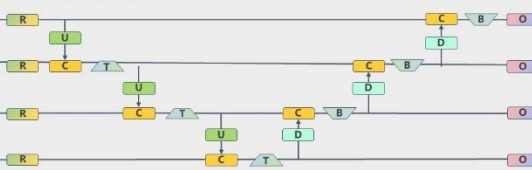
Backbone

YOLOv5
CSPDarknet
(P6)



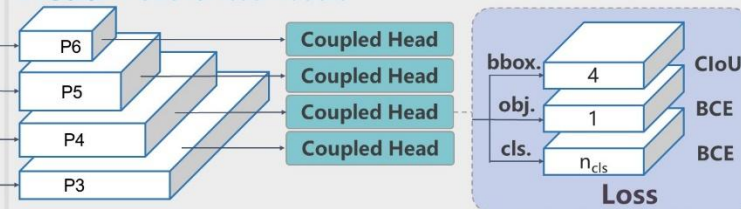
Neck

YOLOv5PAFPN

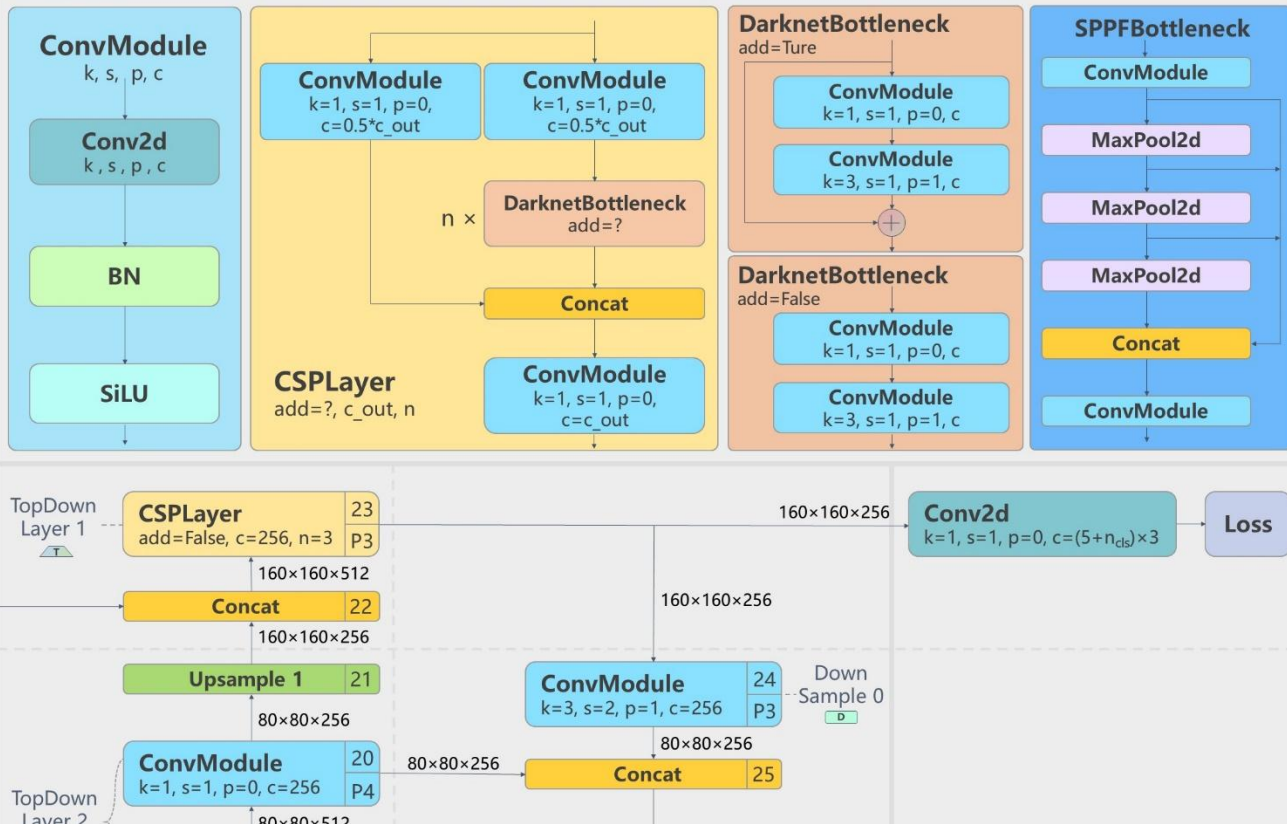


Head

YOLOv5HeadModule



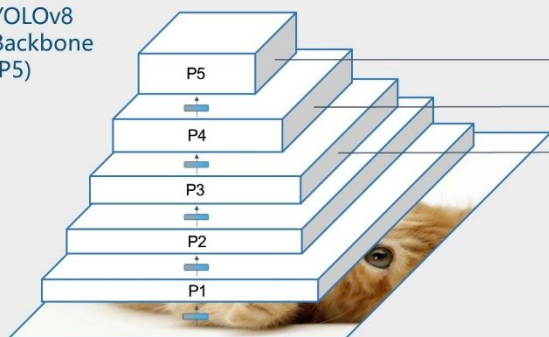
Details



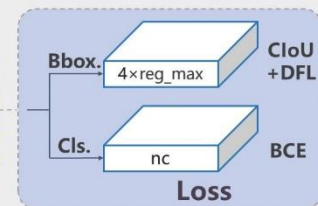
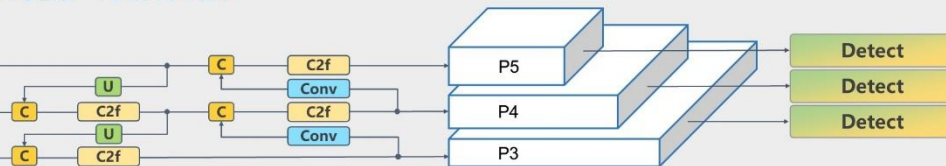
YOLOv8

Backbone

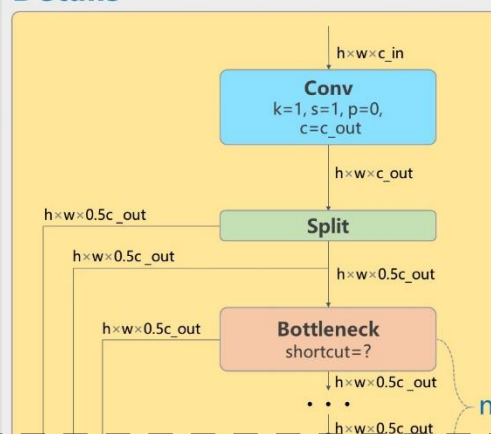
YOLOv8
Backbone
(P5)



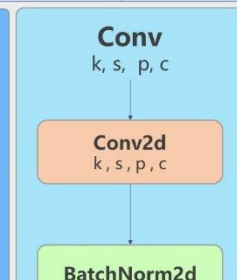
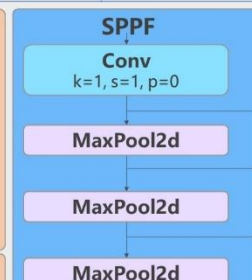
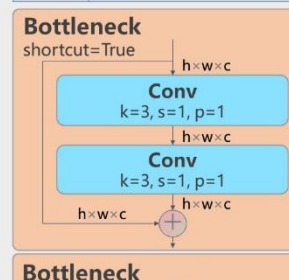
Head YOLOv8Head



Details



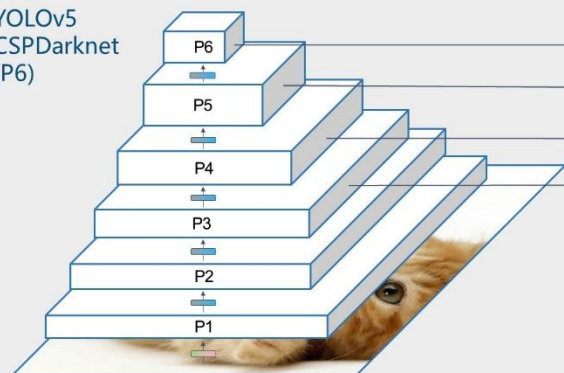
model	d (depth_multiple)	w (width_multiple)	r (ratio)
n	0.33	0.25	2.0
s	0.33	0.50	2.0
m	0.67	0.75	1.5
l	1.00	1.00	1.0
x	1.00	1.25	1.0



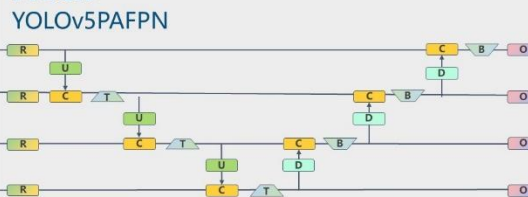
YOLOv5-I

Backbone

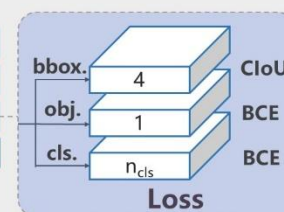
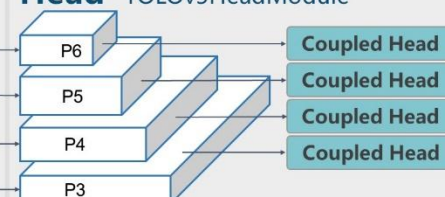
YOLOv5
CSPDarknet
(P6)



Neck YOLOv5PAFPN



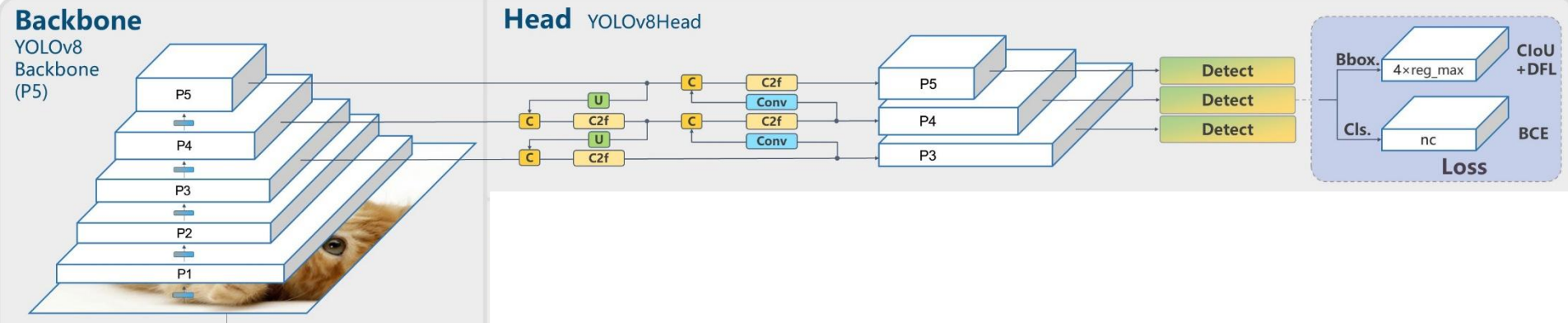
Head YOLOv5HeadModule



YOLO v8



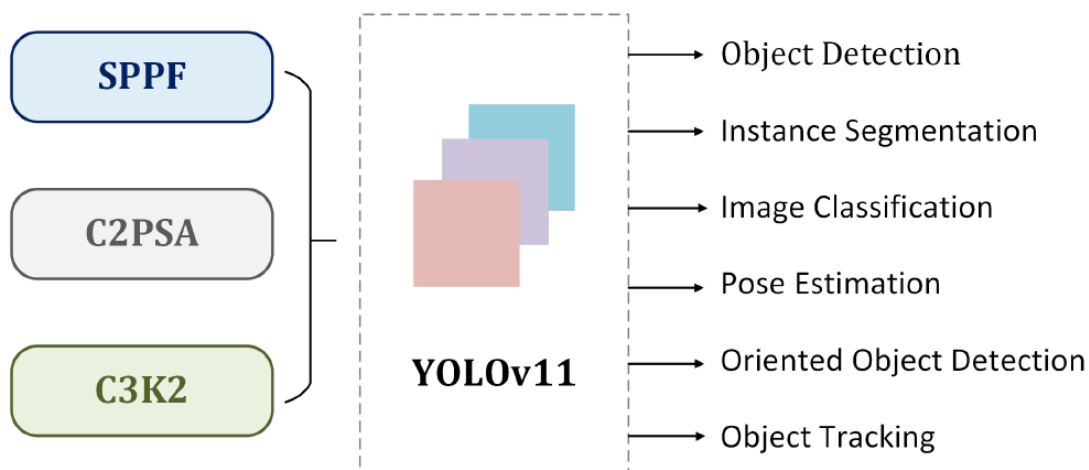
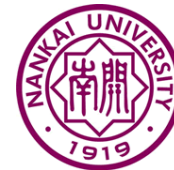
YOLOv8



Metric	YOLOv5	YOLOv8
mAP@0.5	50.5%	55.2%
Inference Time	30 ms/image	25 ms/image
Training Time	12 hours	10 hours
Model Size	14 MB	12 MB

YOLO v11

<https://github.com/ultralytics/ultralytics>
<https://arxiv.org/abs/2410.17725v1>



Classify



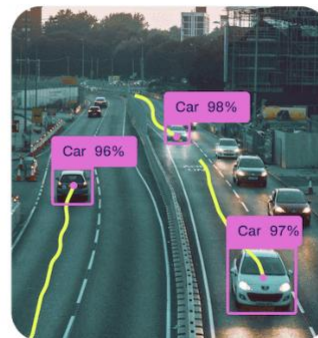
Detect



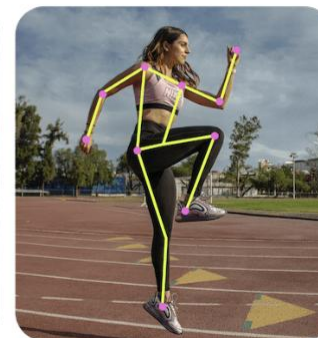
Segment



Track

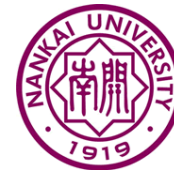


Pose



YOLO v11

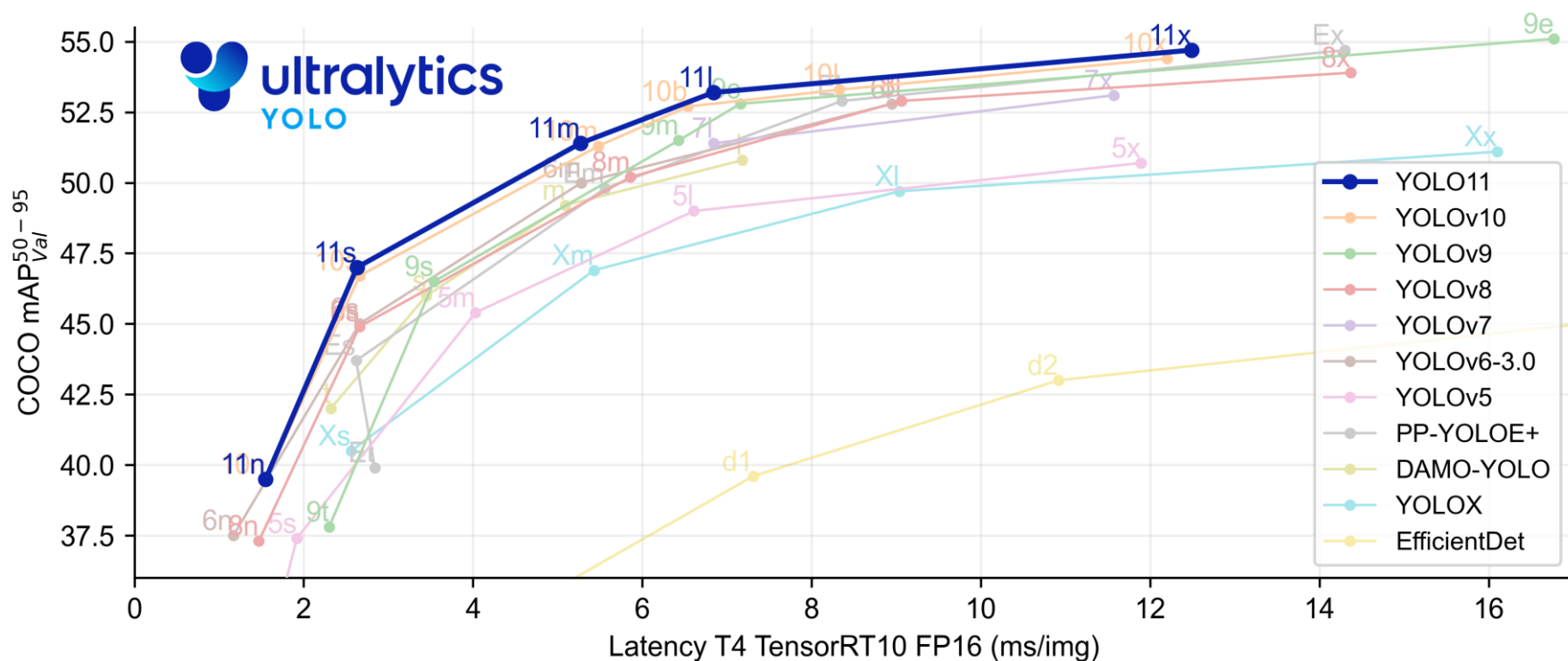
<https://github.com/ultralytics/ultralytics>



Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Model	size (pixels)	mAP ^{val} 50-95	mAP ^{val} 50	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5n	640	28.0	45.7	45	6.3	0.6	1.9	4.5
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5
YOLOv5m	640	45.4	64.1	224	8.2	1.7	21.2	49.0
YOLOv5l	640	49.0	67.3	430	10.1	2.7	46.5	109.1

2) YOLO系列



主要内容

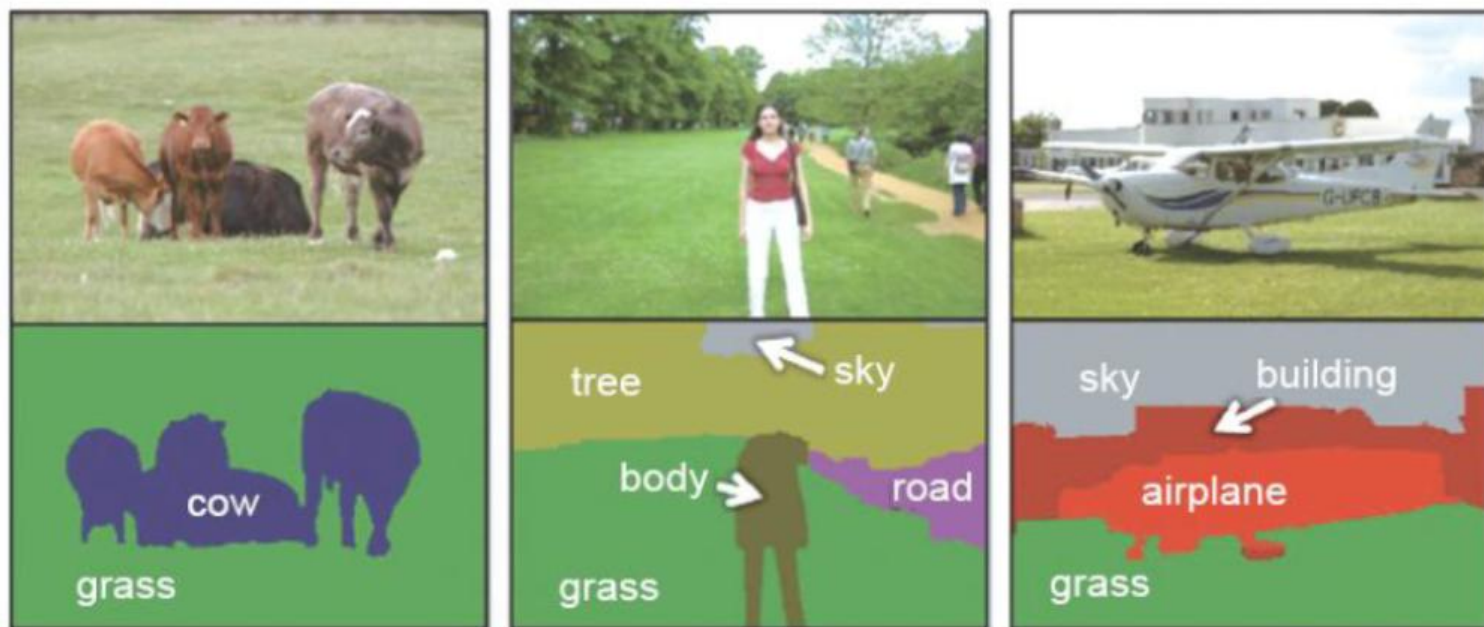
- 第六章 卷积神经网络
- 第七章 图像分类
- 第八章 目标检测与图像分割
 - 8.1 目标检测
 - 8.2 语义分割
 - 8.3 实例分割

图像分割



➤ 语义分割

- 将图片中的所有像素进行分类（包括背景），不区分具体目标，仅做像素级分类。例如，将图上面一行图片进行语义分割的结果为下面一行图片



a) 语义分割

图像分割

➤ 实例分割

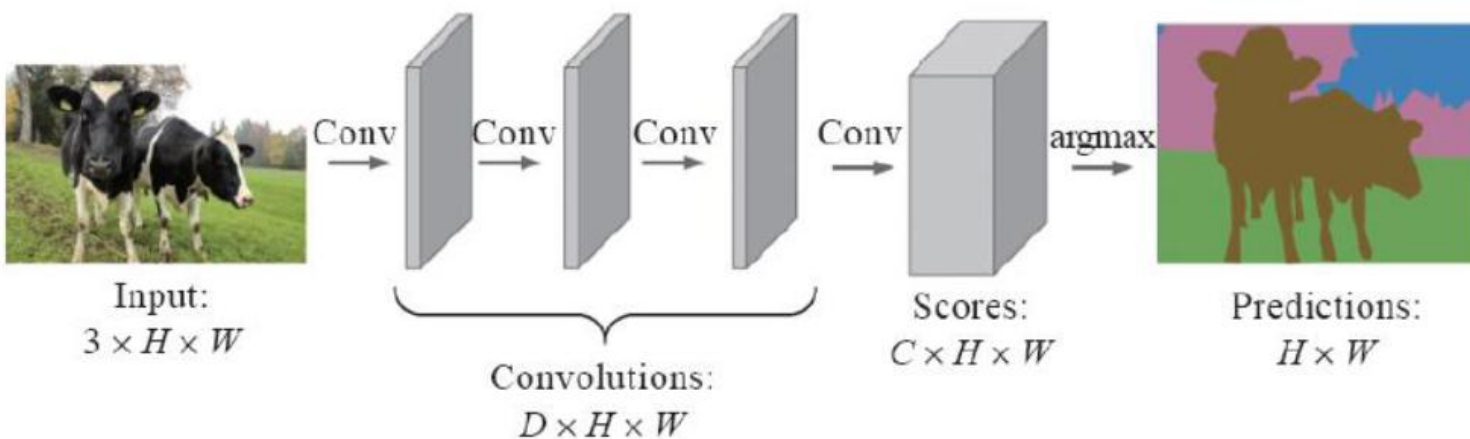
对于有多个目标的图片，对每个目标完成像素级的分类，并区分每一个目标（即区分同一个类别但属于不同的目标）



8.2 语义分割

➤ 8.2.1 FCN(Fully Convolutional Networks)

- 对原图的每个像素进行分类，那么将输出层的每一个像素点当作分类任务做一个Softmax
- 对于一张 $W \times H \times 3$ 的图片，中间经过若干层卷积，卷积的kernel大小为 $W \times H$ ，最终通过一个 $W \times H \times C$ （ C 为之前定义好的类别个数）的Softmax对原图的每一个像素进行分类

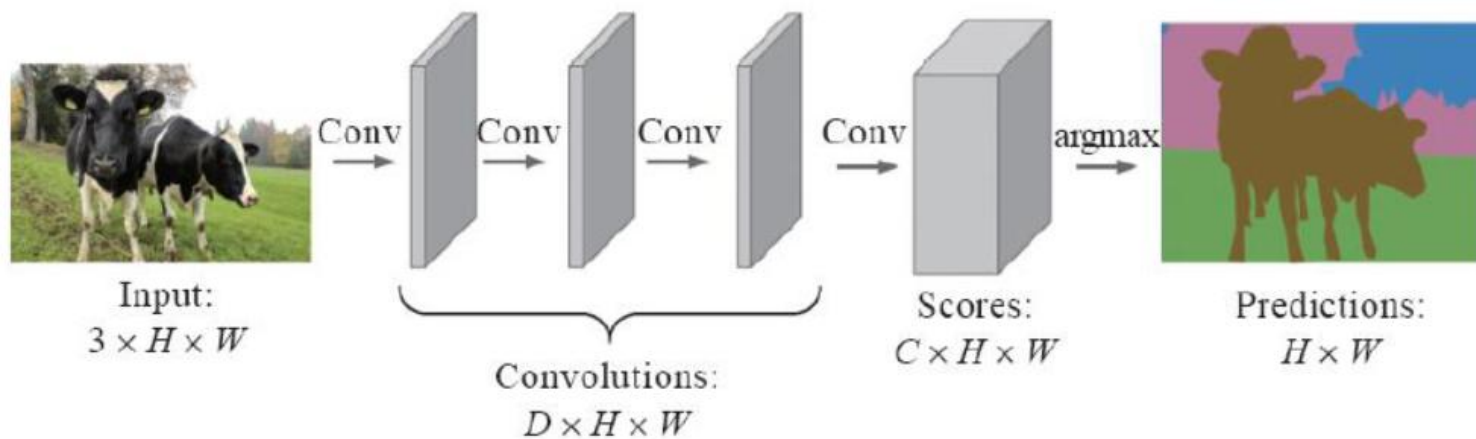
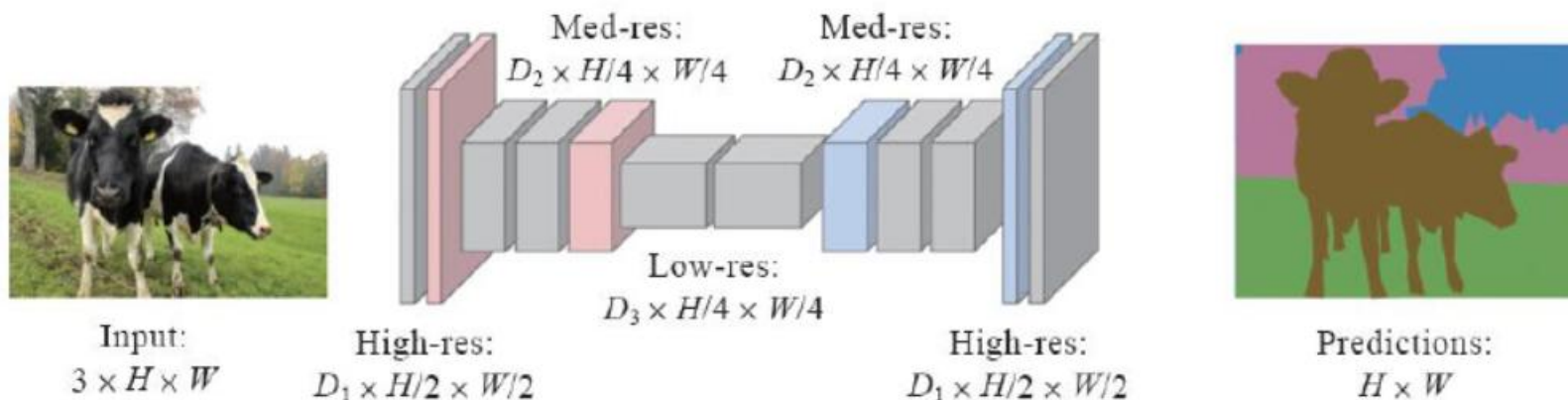


8.2.1 FCN

- **FCN缺点：**中间卷积层尺度太大，内存和计算量的消耗也非常大
- 在2015年，Long、Shelhamer、Darrell和Noh等人提出了在卷积神经网络内部使用下采样和上采样结合的方式实现图片的语义分割
- 下采样主要是通过我们之前学习过的Pooling（池化）和调整卷积的stride（步幅）来实现的
- 上采样的过程与下采样相反，主要包括Unpooling（反池化）和Deconvolution（反卷积）两种方式

8.2.1 FCN

➤ 改良后的CNN语义分割网络结构



8.2.1 FCN

➤ Unpooling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2×2

Output: 4×4

a) 最近邻

“Bed of Nails”

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2×2

Output: 4×4

b) 固定位置

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8



5	6
7	8



Rest of the network

Use positions from pooling layer

1	2
3	4



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Input: 4×4

Output: 2×2

Input: 2×2

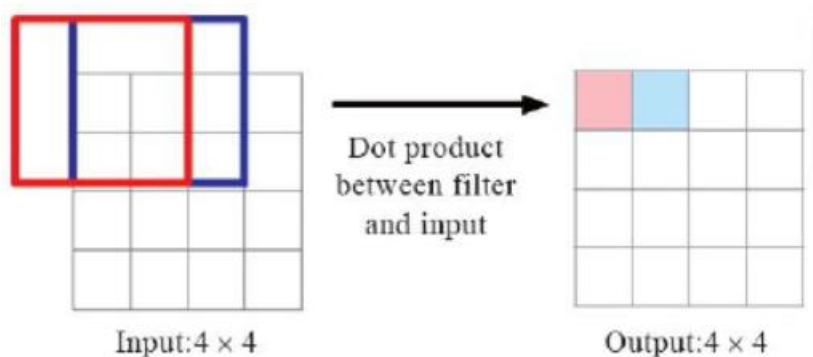
Output: 4×4

c) Max Unpooling

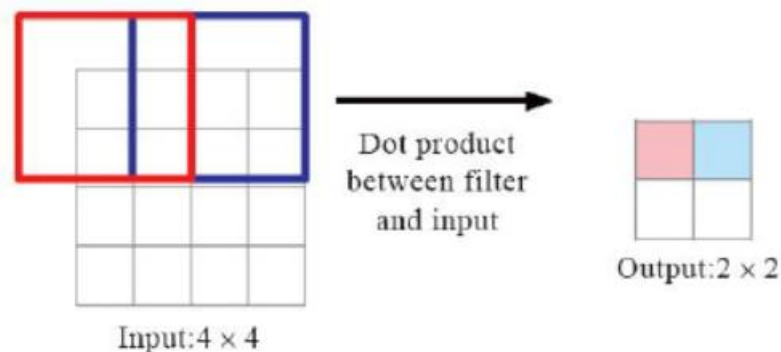
8.2.1 FCN

➤ Deconvolution

- 反卷积是卷积的一种反向操作，可以通过学习得到
- 对于一个 4×4 的图片做kernel为 3×3 、pad为1、stride分别为1和2的卷积



a) kernel 为 3×3 、stride 为 1、pad 为 1 的卷积

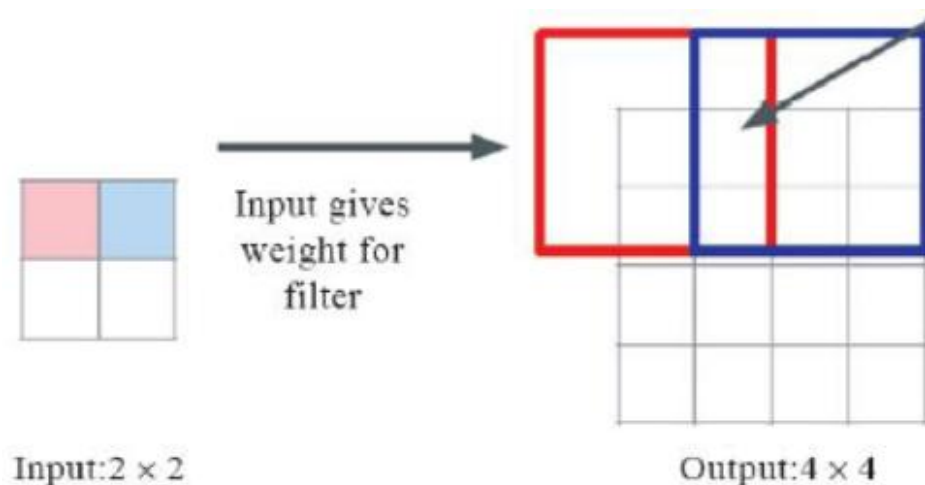


b) kernel 为 3×3 、stride 为 2、pad 为 1 的卷积

8.2.1 FCN

➤ Deconvolution

- 对于一张 2×2 的图片，做一个kernel为 3×3 、pad为1、stride为2的反卷积操作
- 第一步产生的红框与第二步产生的蓝框有交叠，那么交叠的部分该如何计算呢？答案是对两次计算结果进行叠加

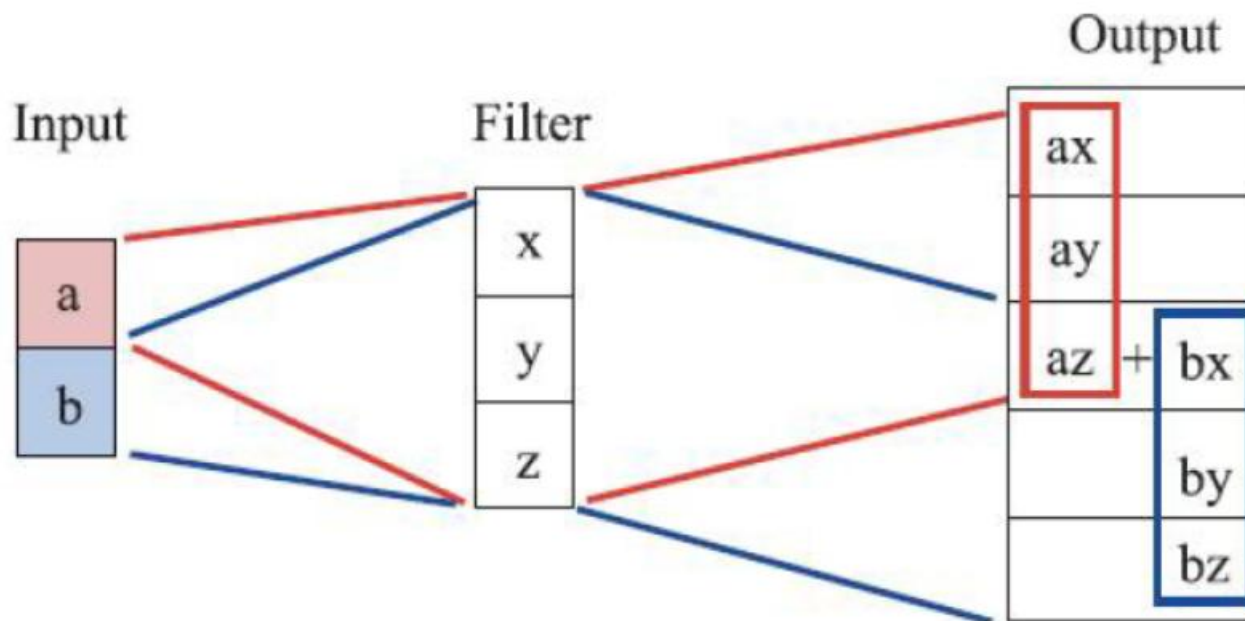


c) kernel 为 3×3 、stride 为 2、pad 为 1 的反卷积

8.2.1 FCN

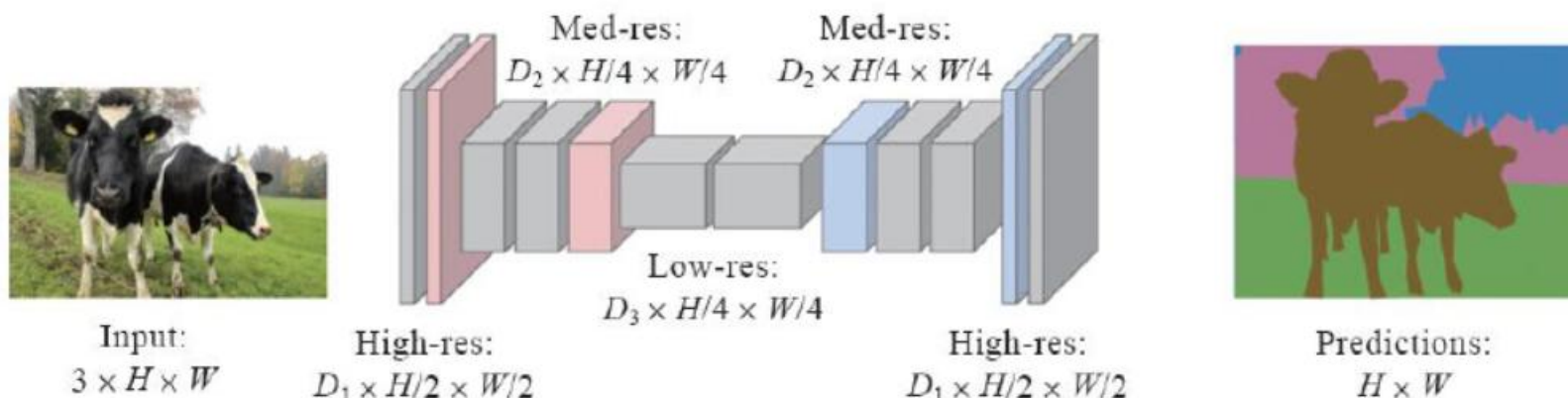
➤ Deconvolution

- 将问题简到1维，kernel为3、stride为2的1维反卷积计算过程的细节



8.2.1 FCN

- 不论是Uppooling方式还是Deconvolution的方式，网络结构的关键层都是以卷积的方式进行操作，不涉及类似全连接这种操作，因此我们通常称这种网络为**全卷积网络**（Full Connected Network, FCN）
- 全卷积网络可用作语义分割，最经典的莫过于2015年夺得CVPR最佳论文的《Fully Convolutional Networks for Semantic Segmentation》，其思路主要就是下采样加上采样的方法



8. 2. 2 UNet



2015:

U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

Computer Science Department and BIOS Centre for Biological Signalling Studies,
University of Freiburg, Germany

2016:

3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation

Özgün Çiçek^{1,2}, Ahmed Abdulkadir^{1,4}, Soeren S. Lienkamp^{2,3}, Thomas Brox^{1,2}, and Olaf Ronneberger^{1,2,5}

¹ Computer Science Department, University of Freiburg, Germany

² BIOS Centre for Biological Signalling Studies, Freiburg, Germany

³ University Hospital Freiburg, Renal Division, Faculty of Medicine, University of Freiburg, Germany

⁴ Department of Psychiatry and Psychotherapy, University Medical Center Freiburg, Germany

⁵ Google DeepMind, London, UK

8. 2. 2 UNet



2019:

nature | **methods**

BRIEF COMMUNICATION

<https://doi.org/10.1038/s41592-018-0261-2>

Corrected: Author Correction

U-Net: deep learning for cell counting, detection, and morphometry

Thorsten Falk^{1,2,3,20}, Dominic Mai^{1,2,4,16,20}, Robert Bensch^{1,2,17,20}, Özgün Çiçek¹, Ahmed Abdulkadir^{1,5}, Yassine Marrakchi^{1,2,3}, Anton Böhm¹, Jan Deubner^{6,7}, Zoe Jäckel^{6,7}, Katharina Seiwald⁶, Alexander Dovzhenko^{8,18}, Olaf Tietz^{8,18}, Cristina Dal Bosco⁸, Sean Walsh^{8,18}, Deniz Saltukoglu^{2,9,10,11}, Tuan Leng Tay^{7,12,13}, Marco Prinz^{2,3,12}, Klaus Palme^{2,8}, Matias Simons^{2,9,10,14}, Ilka Diester^{6,7,15}, Thomas Brox^{1,2,3,7} and Olaf Ronneberger^{1,2,19*}

¹Department of Computer Science, Albert-Ludwigs-University, Freiburg, Germany. ²BIOS Centre for Biological Signalling Studies, Freiburg, Germany.

³CIBSS Centre for Integrative Biological Signalling Studies, Albert-Ludwigs-University, Freiburg, Germany. ⁴Life Imaging Center, Center for Biological Systems Analysis, Albert-Ludwigs-University, Freiburg, Germany. ⁵University Hospital of Old Age Psychiatry and Psychotherapy, University of Bern, Bern, Switzerland. ⁶Optophysiology Lab, Institute of Biology III, Albert-Ludwigs-University, Freiburg, Germany. ⁷BrainLinks-BrainTools, Albert-Ludwigs-University, Freiburg, Germany. ⁸Institute of Biology II, Albert-Ludwigs-University, Freiburg, Germany. ⁹Center for Biological Systems Analysis (ZBSA), Albert-Ludwigs-University, Freiburg, Germany. ¹⁰Renal Division, University Medical Centre, Freiburg, Germany. ¹¹Spemann Graduate School of Biology and Medicine (SGBM), Albert-Ludwigs-University, Freiburg, Germany. ¹²Institute of Neuropathology, University Medical Centre, Freiburg, Germany.

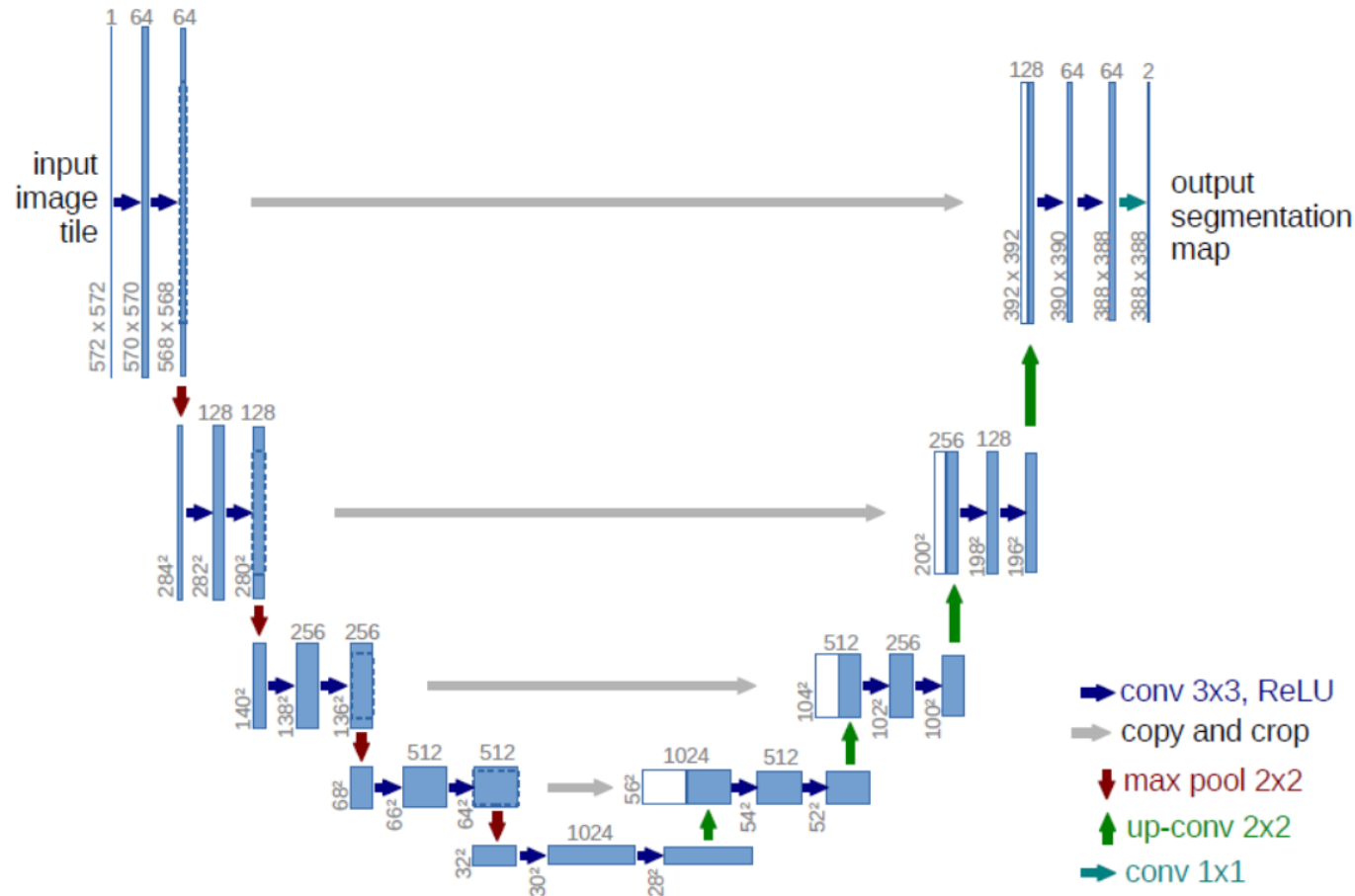
¹³Institute of Biology I, Albert-Ludwigs-University, Freiburg, Germany. ¹⁴Paris Descartes University-Sorbonne Paris Cité, Imagine Institute, Paris, France.

¹⁵Bernstein Center Freiburg, Albert-Ludwigs-University, Freiburg, Germany. ¹⁶Present address: SICK AG, Waldkirch, Germany. ¹⁷Present address: ANavS GmbH, München, Germany. ¹⁸Present address: ScreenSYS GmbH, Freiburg, Germany. ¹⁹Present address: DeepMind, London, UK. ²⁰These authors contributed equally: Thorsten Falk, Dominic Mai, Robert Bensch. *e-mail: ronneber@informatik.uni-freiburg.de

8. 2. 2 UNet

网络结构

- 下采样后经过2次卷积后再次下采样
- 上采样则使用反卷积的方式，并与对应大小的下采样特征层进行连接，然后经过2次卷积后再反卷积



8.2.2 UNet

➤ 损失函数

- 最终特征图上的像素级softmax+交叉熵损失函数

$$E = \sum_{\mathbf{x} \in \Omega} w(\mathbf{x}) \log(p_{l(\mathbf{x})}(\mathbf{x}))$$

- $p_{l(\mathbf{x})}$: softmax, l : 像素标签(前景or背景)
- $w(\mathbf{x})$: 像素点权重

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)$$

- W_c : 平衡类别比例的权值, d_1 是像素点到距离其最近的细胞的距离, d_2 是像素点到距离其第二近的细胞的距离

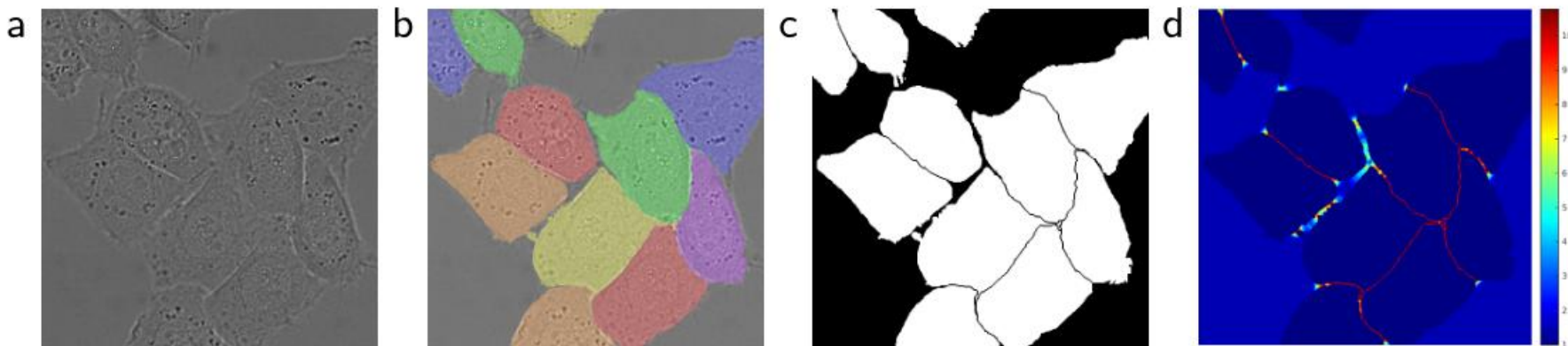
8.2.2 UNet

➤ 损失函数

- $w(\mathbf{x})$: 像素点权重

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp \left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2} \right)$$

- w_c : 平衡类别比例的权值, d_1 是像素点到距离其最近的细胞的距离, d_2 是像素点到距离其第二近的细胞的距离



Hela细胞原图

Ground Truth

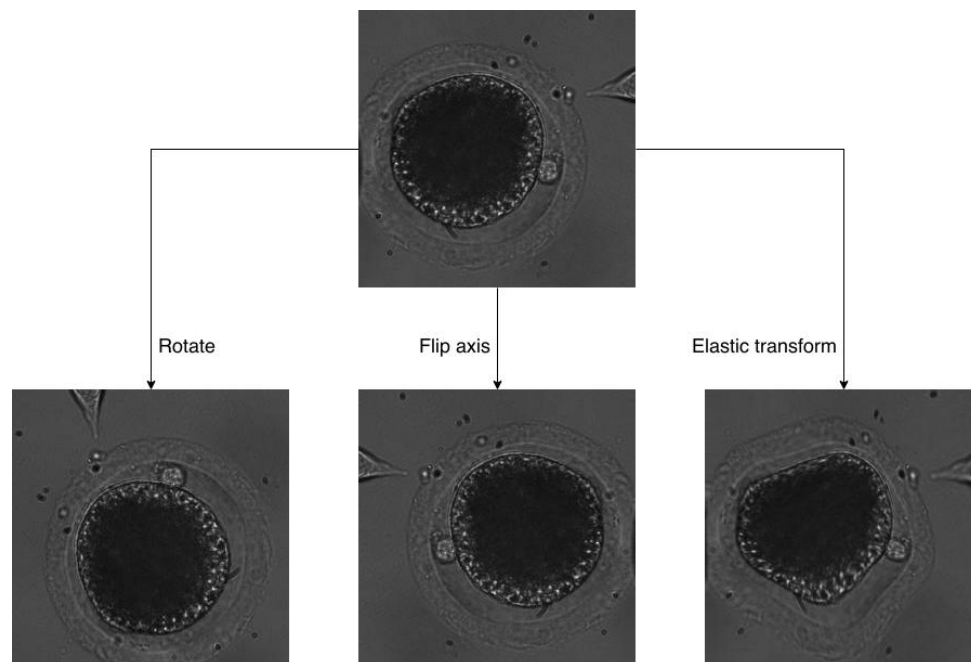
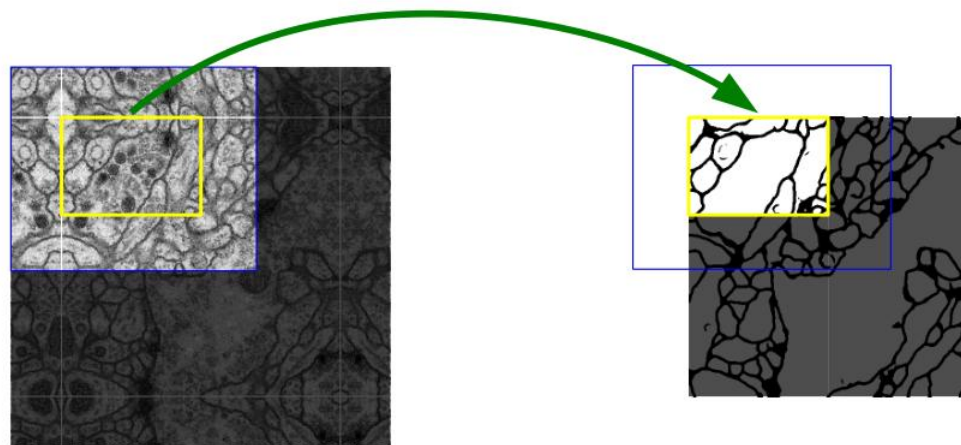
分割掩膜

像素级损失权重

8. 2. 2 UNet

➤ 数据增强/数据预处理

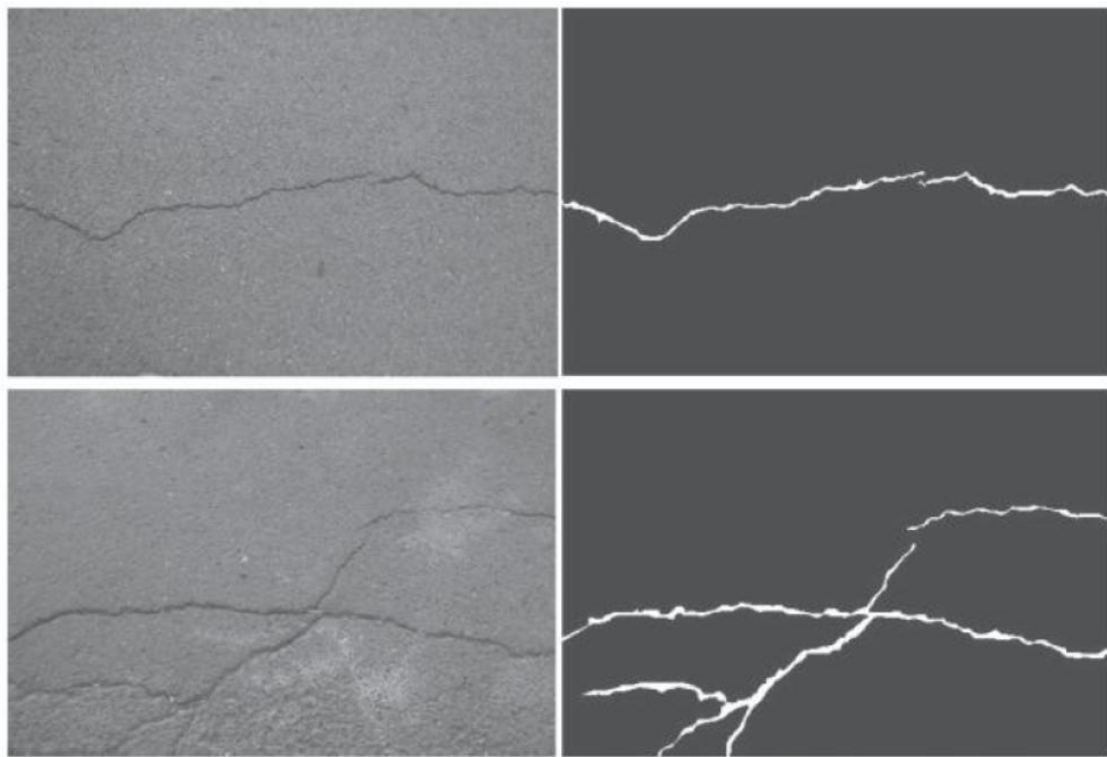
- 通过移动窗口的方式将一张图像切分成多个子图(Overlap-tile strategy)
- 旋转、翻转，弹性形变



8. 2. 2 UNet

- UNet网络结构很简单，因此对于小样本量数据集有很好的效果。下面列举一个简单的案例，如何通过U-Net来识别道路上的裂纹
- 数据集：

<https://github.com/cuilimeng/CrackForest-dataset>



8. 2. 2 UNet

➤ Unet实现裂纹分割

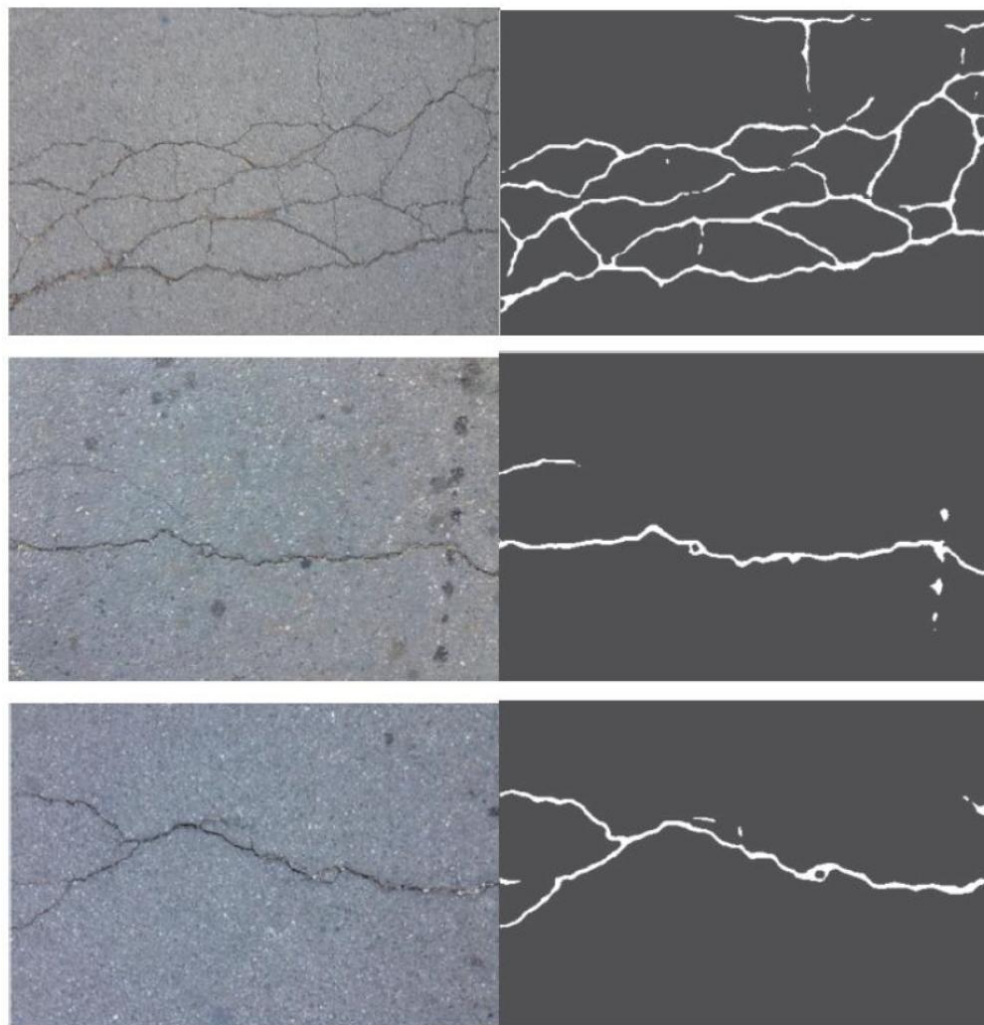
➤ 预测效果

Epoch: 168 Train loss 0.0162, accuracy 0.9938, time 4.64

Validation loss 0.0273, accuracy 0.9903, time 2.26

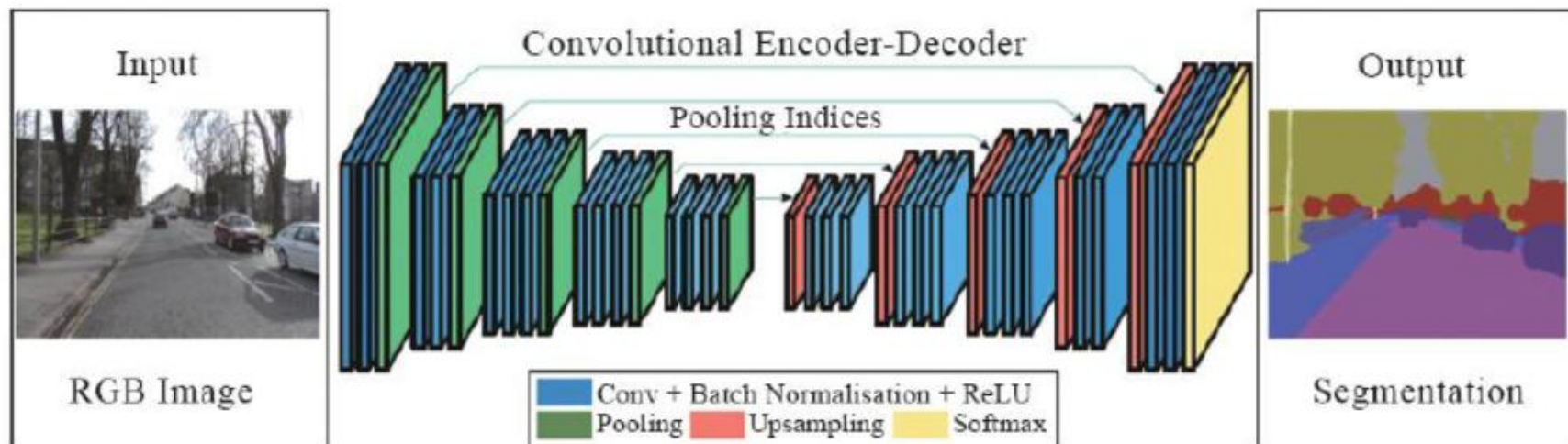
Epoch: 169 Train loss 0.0157, accuracy 0.9941, time 4.67

Validation loss 0.0268, accuracy 0.9910, time 2.26



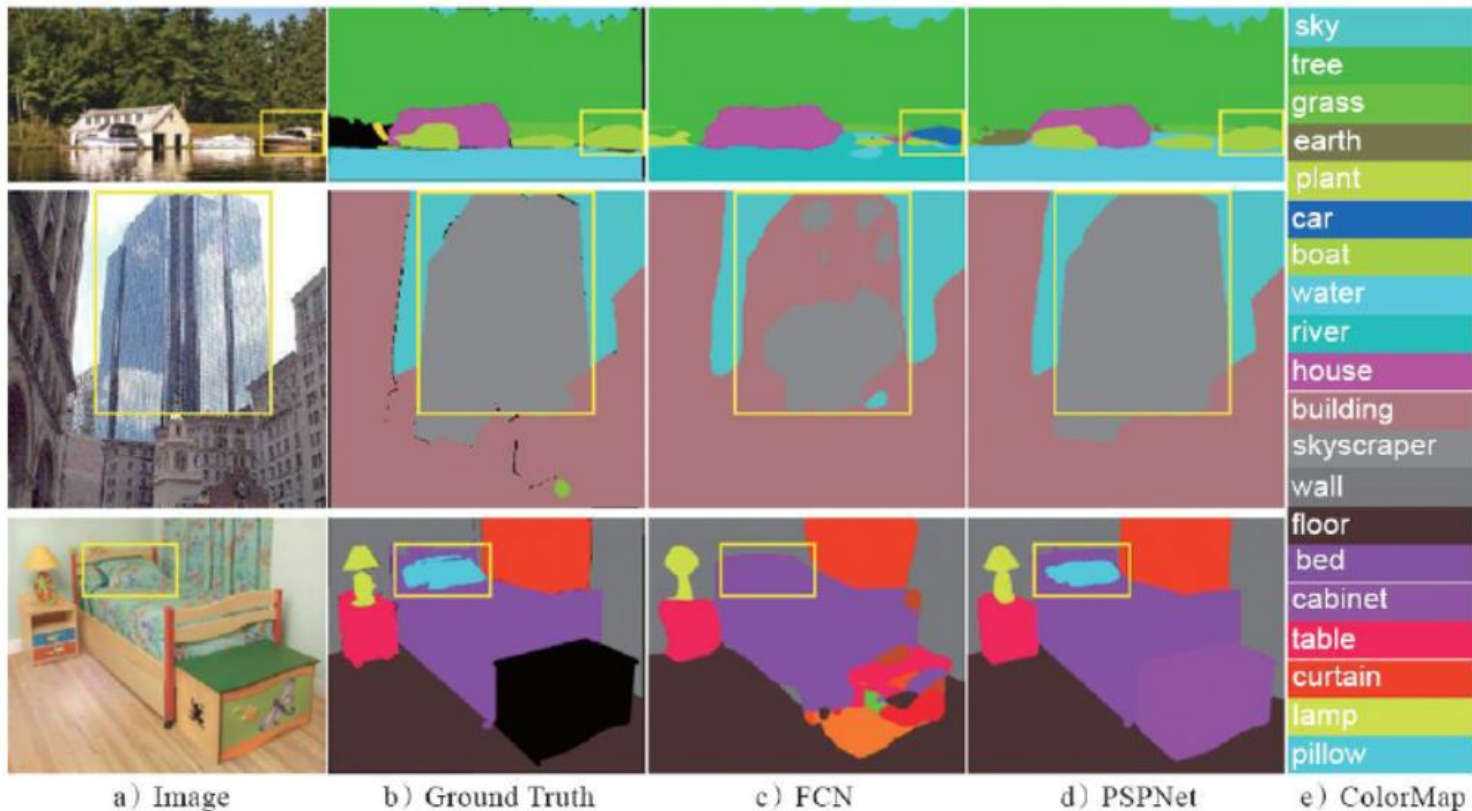
8. 2. 3 SegNet

- 与U-Net结构类似，SegNet也是分割网络，并由编码器、解码器、像素级分类器3个部分组成。
- 考虑到普通卷积神经网络在做下采样的过程中，其空间分辨率有所损失，然而边缘轮廓对分割问题而言比较重要
- 在下采样时获取Max-Pooling的索引，即为每个特征层在每个下采样滑窗上记录Max-Pooling的位置和最大值。在上采样的时候将对应Max-Pooling的索引位置映射回去，之后再通过卷积进行学习



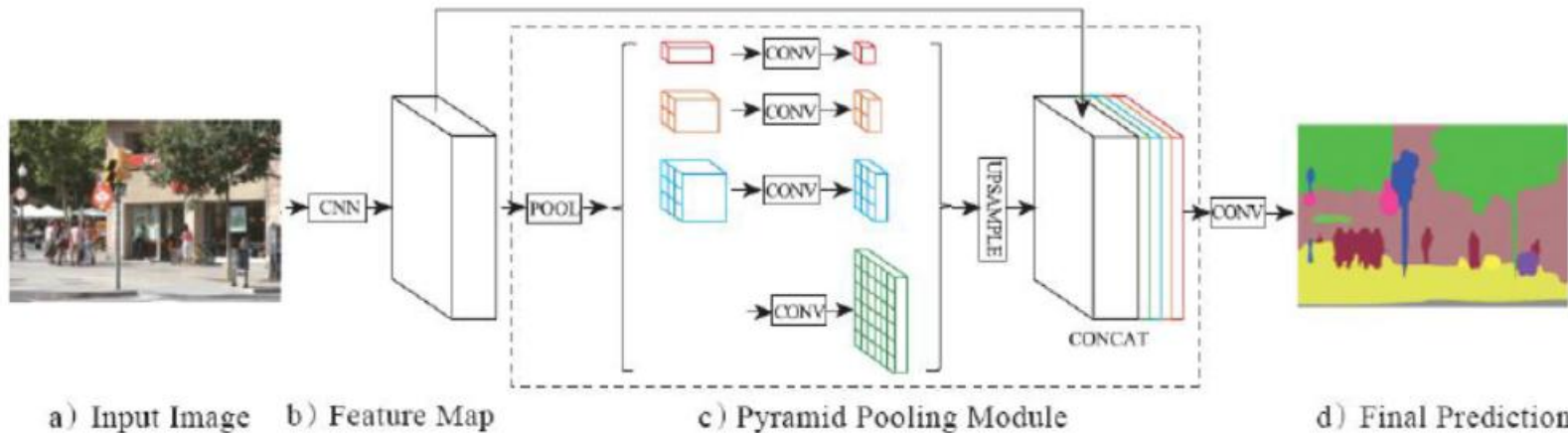
8. 2. 4 PSPNet

- PSPNet在FCN、U-Net、SegNet的基础上，考虑了上下文和局部的信息去做预测，融合了不同尺度的池化模块。



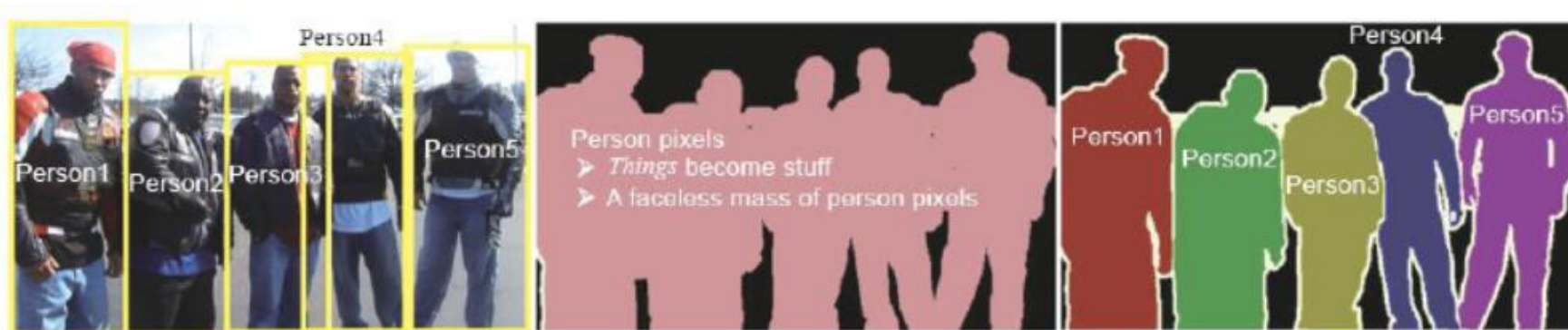
8.2.4 PSPNet

- 对特征层使用4个不同的尺度进行池化
 - 红色部分是全局池化，黄色部分是将特征层分为4块不同的子区域进行池化，后面蓝色、绿色的层以此类推
 - 池化后为了保证不同尺度的权重，通过 1×1 的卷积对特征层降维到 $1/N$ (N 为不同尺度的数量，这里是4)
 - 将 N 个降维后的特征层上采样到原特征层尺寸并与原特征层进行合并，最终形成多尺度融合后的特征层



8.3 实例分割

- 目标检测是将图片中的物体位置检测出来并给出相应的物体类别
- 语义分割是给出每个像素的类别，只区分类别，不区分是否为同一个物体
- 实例分割则是要给出类别并区分出不同的物体



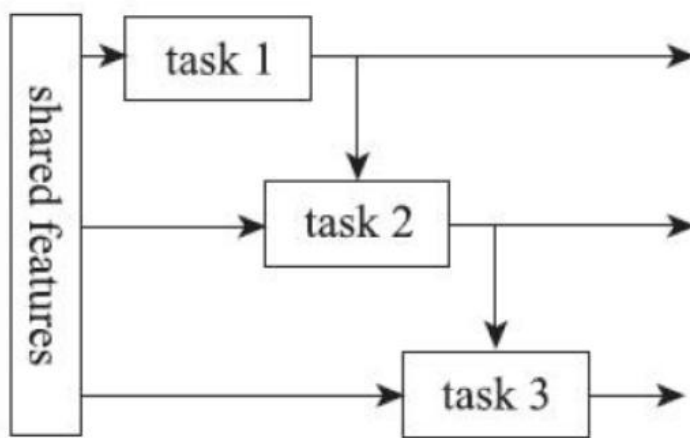
a) 目标检测

b) 语义分割

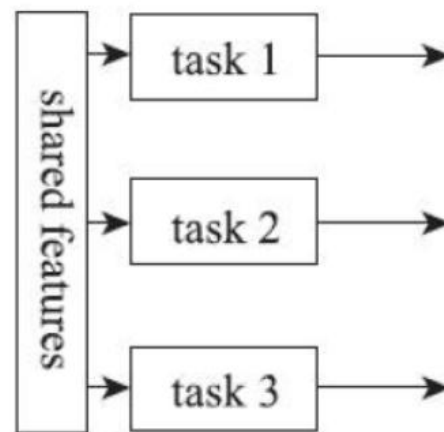
c) 实例分割

8.3 实例分割

- 实例分割需要识别出“每一个物体”，又需要得到像素级别分割的结果，是一个多任务学习的方法
- 多任务学习通常具有两种方式
 - 堆叠式，即将不同的任务通过某种方式连接起来进行训练
 - 扁平式，即每个任务单独训练



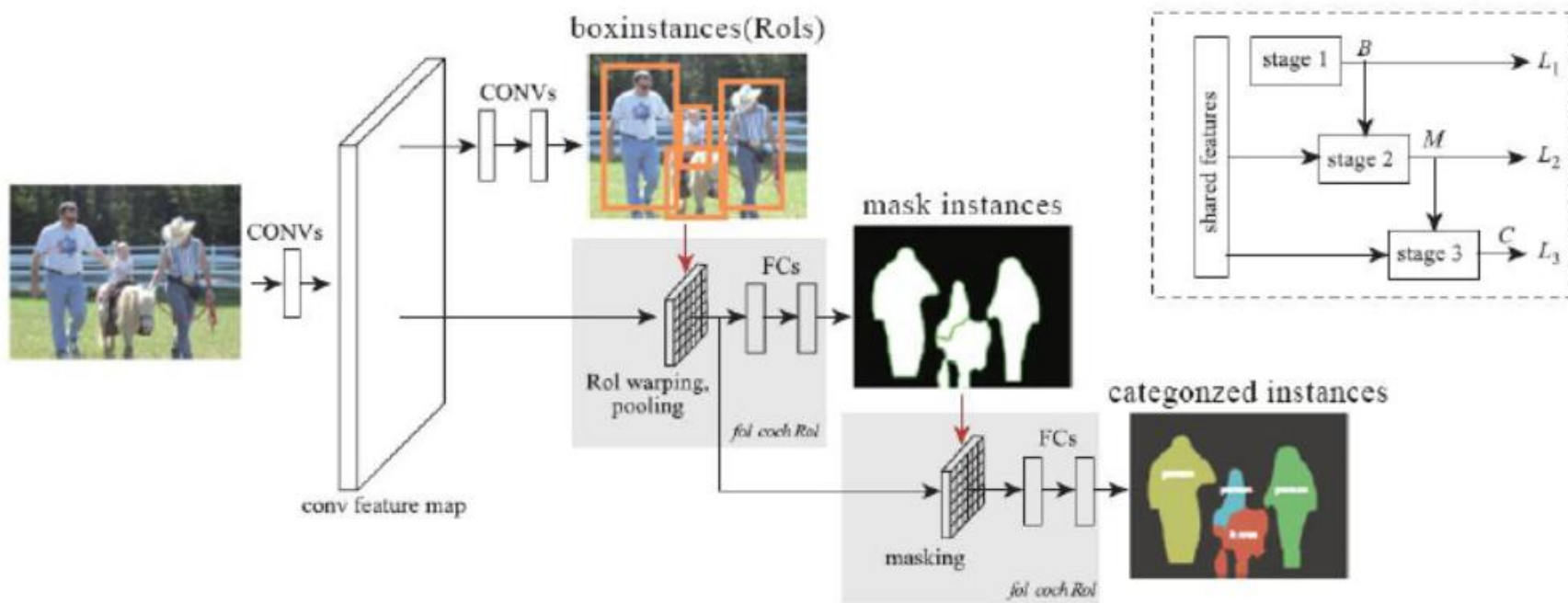
a) 层叠式



b) 扁平式

8.3.1 层叠式

➤ 层叠式实例分割网络结构



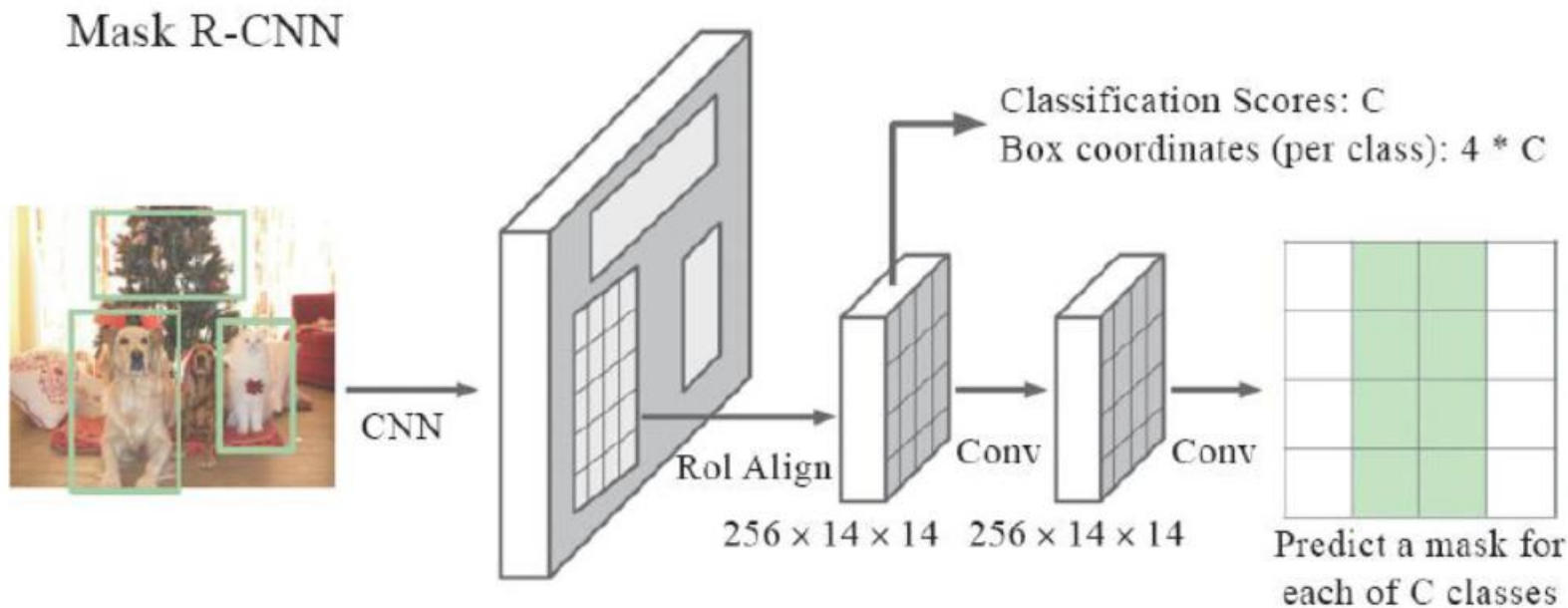
Instance-aware Semantic Segmentation via Multi-task Network Cascades, 2015

8.3.1 层叠式

- 层叠式方法具体可分为3个步骤
 - 1) 检测——使用Faster R-CNN的RPN方法预测所有潜在物体的bbox（这里只做二分类区分是不是物体）。
 - 2) 分割——将步骤1) 的ROI压缩到指定尺寸（论文中是 $14*14*128$ ），通过两个全连接层（用来降维）后预测每个像素点是不是物体。
 - 3) 分类——将步骤1) 和步骤2) 的结果统一到一个尺寸做元素级点乘，再过两个全连接层后对每个ROI进行分类预测。

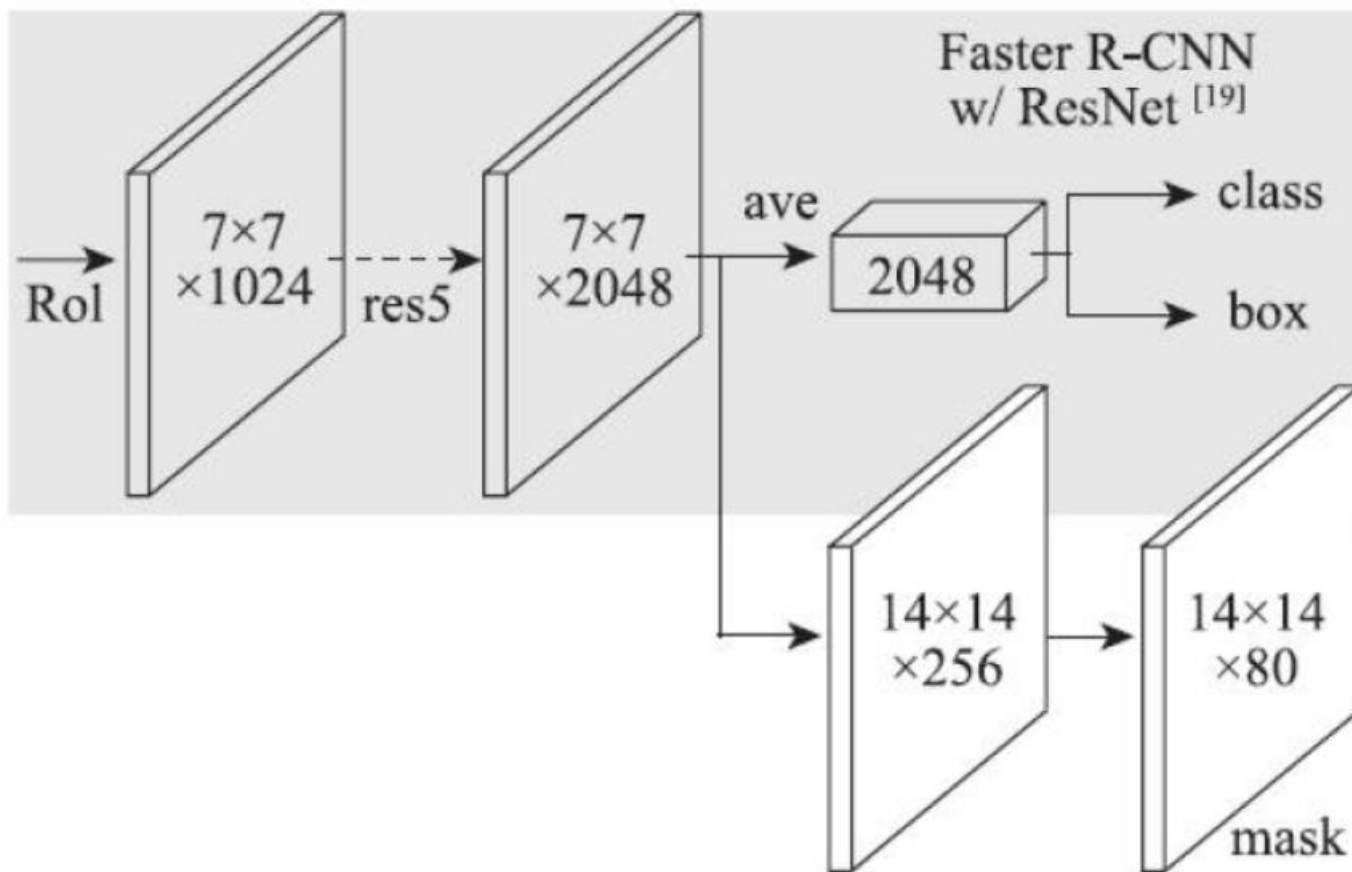
8.3.2 扁平式

- 由于目标检测的结果既有紧贴着目标的矩形框，又有矩形框里目标的类别，那么是否可以基于目标检测的结果，直接在上面做FCN实现像素级分割呢？
- Mask-RCNN:在目前业界公认的效果较好的检测模型 Faster RCNN基础上添加第三个分割分支



8.3.2 扁平式

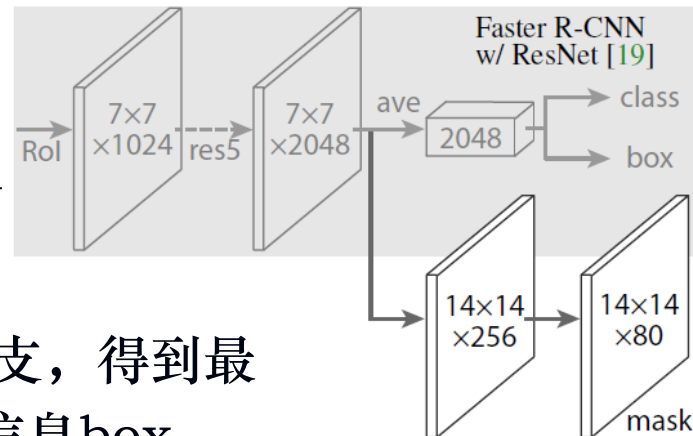
➤ Mask-RCNN的head设计



Mask R-CNN

Mask R-CNN预测阶段的具体步骤

1. 将图像输入网络，通过Fast R-CNN分支，得到最终预测的目标类别信息class和包围框信息box
2. 将目标包围框信息box提供给Mask分支，预测得到该目标对所有类别的mask信息
3. 根据目标类别信息class，将对应该类别的mask信息提取出来(shape为 14×14 ，数值在0到1之间)
4. 利用双线性插值将mask缩放到预测目标包围框大小
5. 通过设定阈值将mask转换为二值图，得到前景与背景

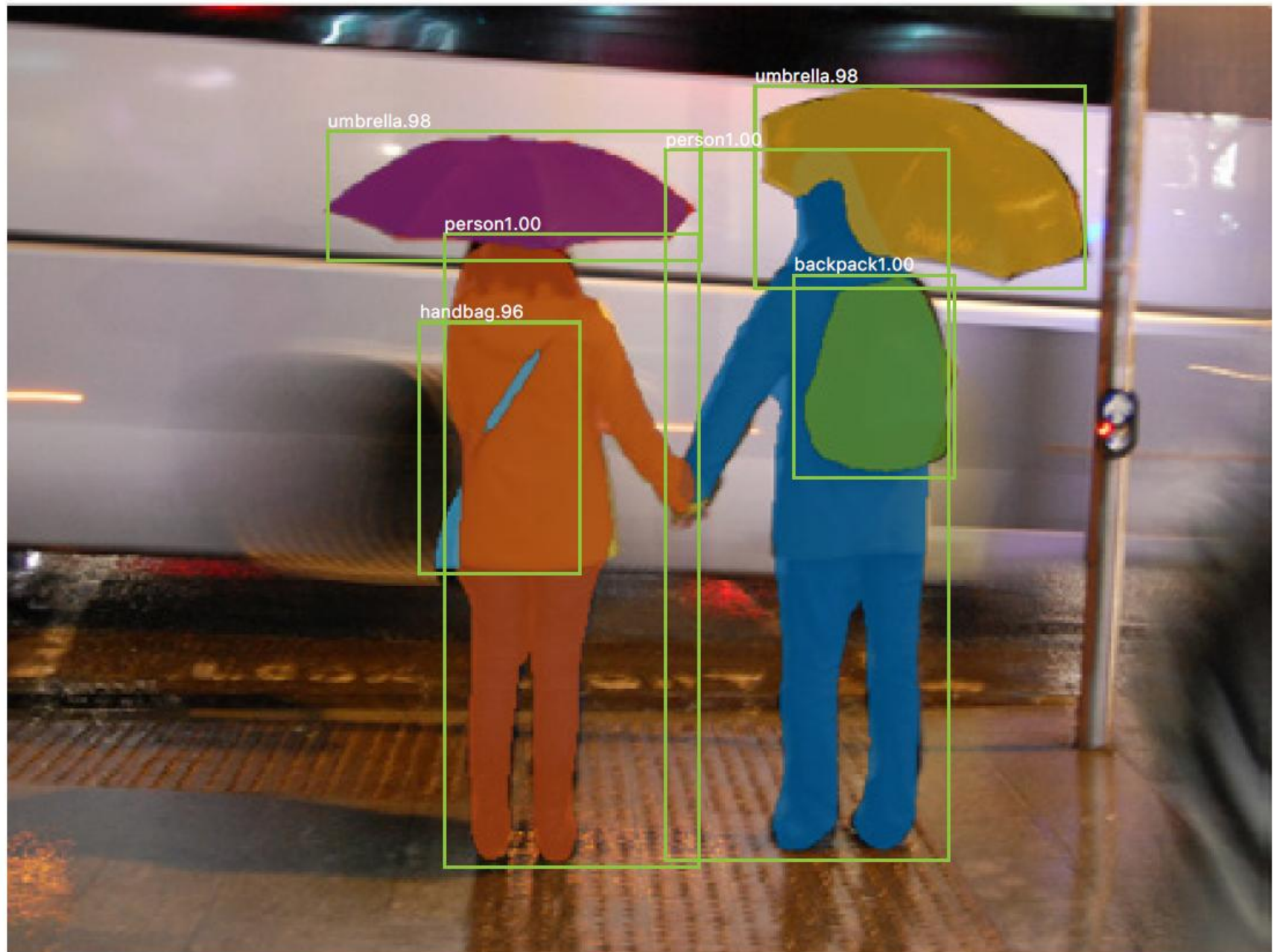


Mask R-CNN



➤ 结果展示





小结

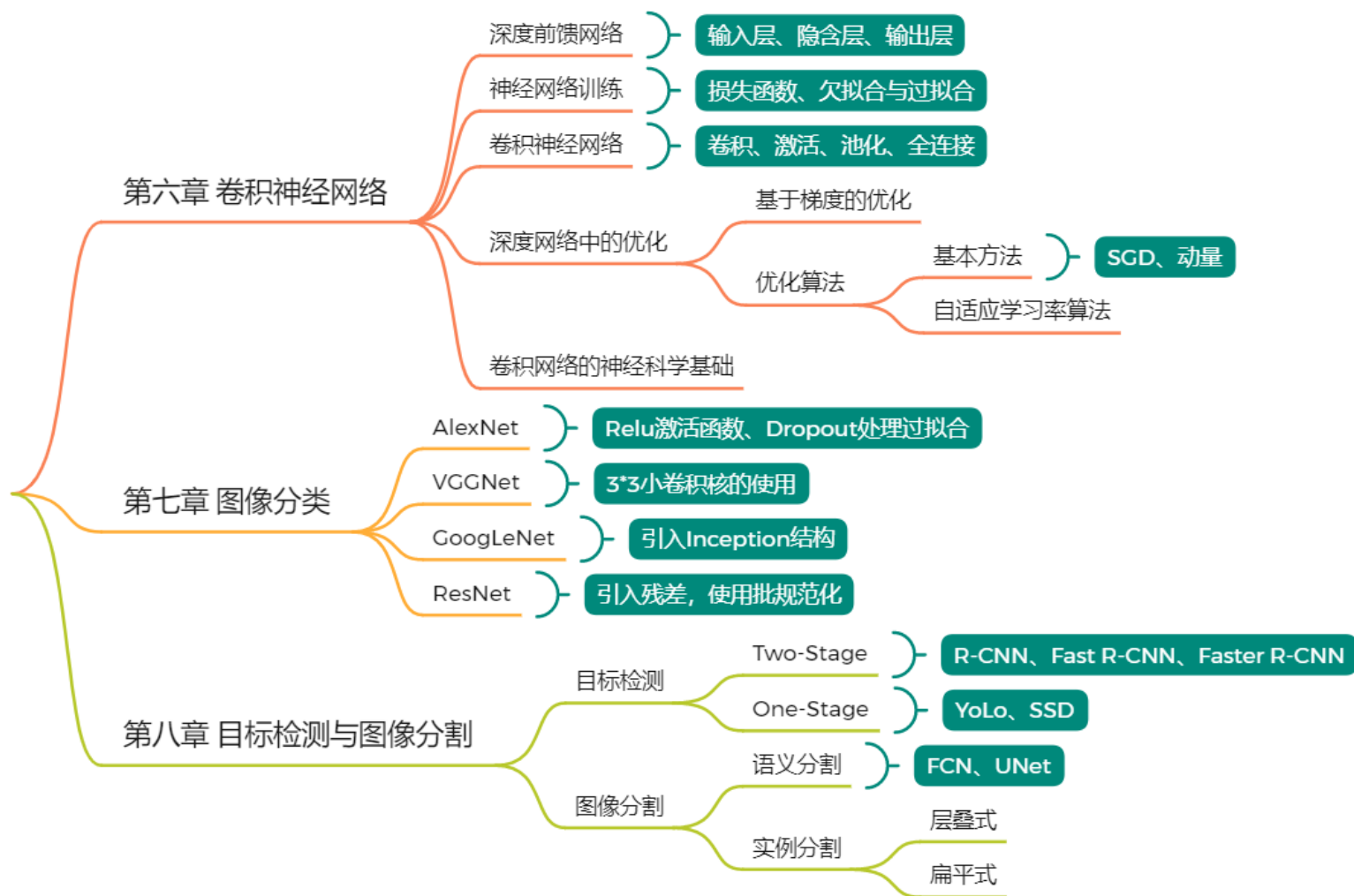


- 语义分割问题的难度介于图像分类和目标检测之间，主要是在网络结构的设计上进行调整，如何设计下采样和上采样的方法和方式是关键。
- 实例分割则通常是在目标检测问题的基础上进行的，流程的设计是关键。

总结



深度学习 图像处理



The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University